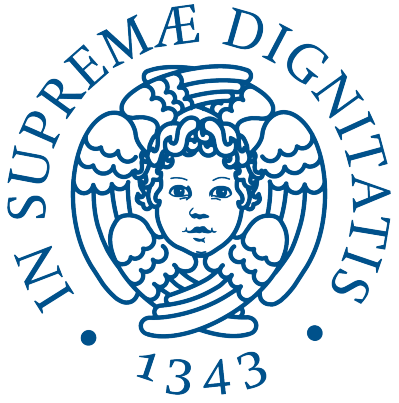

University of Pisa
Master's Degree in Computer Science,
Artificial Intelligence

Computational Mathematics for Learning & Data Analysis (646AA)

Group number: 63
Project number: 25 ML

Members:
Matthew Bernardi
Gabriele Marsili

Academic Year: 2025/2026

Project statement.

25. (P) is so-called *extreme learning*, i.e., a neural network with one hidden layer, $y = w\sigma(W_1 x)$, where the weight matrix for the hidden layer W_1 is a fixed random matrix, $\sigma(\cdot)$ is an elementwise activation function of your choice, and the output weight vector w is chosen by solving a least-squares problem with L_1 regularization $\arg\min_w f(w)$, with $f(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|w\|_1$.

(A1) is *iteratively reweighted least squares*: i.e., an iteration where you solve at each step the least-squares problem $w_{k+1} = \arg\min_w f_k(w)$, $f_k(w) = \frac{1}{2}\|Xw - y\|_2^2 + \lambda\|W_k w\|_2^2$ where W_k is the diagonal matrix with entries $(W_k)_{ii} = |(w_k)_i|^{-1/2}$.

(A2) is an algorithm of the class of the *deflected subgradient methods*.

Contents

Chapter 1

Introduction to the problem

1.1 Extreme Learning Machine

Extreme Learning Machine (ELM) [15] is a machine learning model implemented as a neural network with a single hidden layer, in which:

- $\mathbf{W_1} \in \mathbb{R}^{H \times N}$ is the input-to-hidden weight matrix (where H is the number of nodes in the hidden layer and N is the number of input nodes) that is **randomly generated** and **static** (does not change during training) [15].
- $\sigma(\cdot)$ is the element-wise (nonlinear) activation function applied to the $\mathbf{W_1 x} \in \mathbb{R}^H$ vector, where $\mathbf{x} \in \mathbb{R}^N$ is the input vector. The choice of using an activation function for the hidden layer results, under mild assumptions and with high probability, in the hidden layer feature matrix with full rank when $M \geq H$, since random projections through a nonlinear function produce linearly independent features.
- $\mathbf{w} \in \mathbb{R}^H$ is the hidden-to-output weight vector.
- $\hat{y} = \mathbf{w}^T \sigma(\mathbf{W_1 x}) \in \mathbb{R}$ is the prediction of the model.

Once $\mathbf{W_1}$ is fixed, knowing that $\mathbf{X}_{\text{origin}} \in \mathbb{R}^{M \times N}$ is the matrix of all the inputs, the hidden-layer activations $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}} \mathbf{W}_1^T)$ are constant, and finding $\mathbf{w}$ reduces to solving the Least-Squares problem described in the following section.

1.2 Least-Squares Problem

The definition of the Least Squares Problem [6] in the ELM case is the following:

$$\min_{\mathbf{w}} ||\mathbf{X}\mathbf{w} - \mathbf{y}||_2^2 \tag{1.1}$$

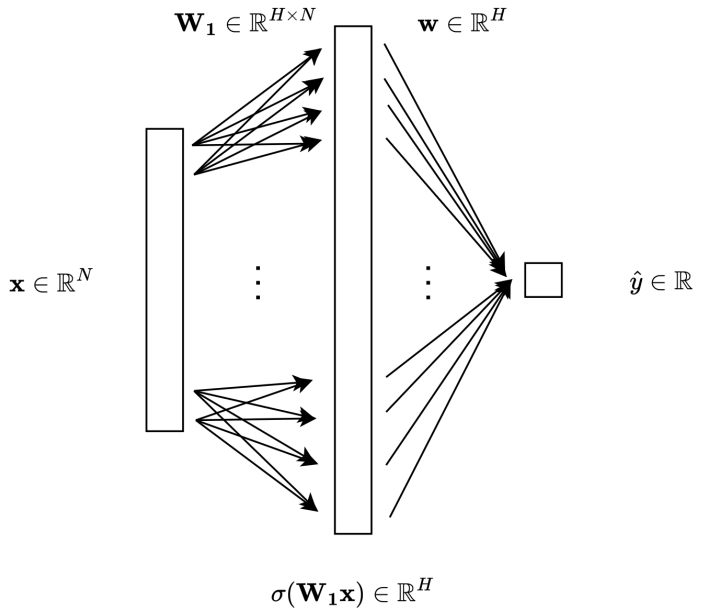

Figure 1.1: Visual representation of an Extreme Learning Machine model.

The Least-Squares problem always admits at least one solution, and the solution is unique if and only if the matrix $\mathbf{X}$ has full column rank. In that case, we can write the optimization problem as follows:

$$\min_{\mathbf{w}} \frac{1}{2}\mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{y}^T\mathbf{X}\mathbf{w} + \frac{1}{2}\mathbf{y}^T\mathbf{y} \tag{1.2}$$

since the gradient of this function is $\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{X}^T\mathbf{y}$, the Hessian matrix is $\mathbf{X}^T\mathbf{X} \succ 0$, which means that the function is strictly convex. In this case, the minimum exists and is unique, and can be found solving the equation:

$$\mathbf{X}^T\mathbf{X}\mathbf{w} - \mathbf{X}^T\mathbf{y} = 0$$

where $\mathbf{X}^T\mathbf{X}$ is square invertible since it's positive definite. The unique solution is $\mathbf{w}_0 = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$. However, adding the L_1 regularization term (Section 1.3) destroys the closed-form structure, requiring iterative methods.

1.3 Regularization Techniques

1.3.1 L_1 regularization

When adding the L_1 regularization term to the Least-Squares, the equation is the following:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \frac{1}{2}||\mathbf{X}\mathbf{w} - \mathbf{y}||_2^2 + \lambda_{\text{LASSO}} \sum_{i}^{H} |w_i| \tag{1.3}$$

This form is particularly nice since it pushes the weights $w_i \to 0$, but the price we have to pay is the non-differentiability of the function where $w_i = 0$, which prevents direct use of gradient-based methods.

Because of that, we can use the Deflected Subgradient Methods (2) and the IRLS algorithm (2), which we will see in the following chapters.

1.4 Objective Function Analysis

Let's break down the function to minimize in two parts:

$$\min_{\mathbf{w}} f(\mathbf{w}) = \underbrace{\frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2}_{g(\mathbf{w})} + \underbrace{\lambda_{\text{LASSO}}\|\mathbf{w}\|_1}_{h(\mathbf{w})}$$

It is useful to analyse the mathematical properties of $f(\mathbf{w})$, as they determine which algorithms can be applied.

1.4.1 Convexity

The mathematical properties of the objective function $f(\mathbf{w})$ are heavily dependent on the matrix $\mathbf{X}$. As we establish below, a full-rank matrix provides the strictly convex structure necessary to guarantee a unique solution.

Proposition 1.1. *If $\mathbf{X} \in \mathbb{R}^{M \times H}$ has full column rank, then $f(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{LASSO}\|\mathbf{w}\|_1$ is strictly convex and admits a unique global minimizer $\mathbf{w}^*$ [2, p. 73].*

Proof. **Strict convexity.** Since $\mathbf{X}$ has full column rank, for any $\mathbf{v} \neq \mathbf{0}$ we have $\mathbf{X}\mathbf{v} \neq \mathbf{0}$, so $\mathbf{v}^T(\mathbf{X}^T\mathbf{X})\mathbf{v} = \|\mathbf{X}\mathbf{v}\|_2^2 > 0$. Hence the Hessian of g is $\nabla^2 g = \mathbf{X}^T\mathbf{X} \succ 0$, making g strictly convex. h is convex as a non-negative multiple of a norm. The sum of a strictly convex function and a convex function is strictly convex [12, Slides 21–24], so f is strictly convex.

Existence and uniqueness of the minimizer. Since $\mathbf{X}^T\mathbf{X} \succ 0$, g is coercive: $g(\mathbf{w}) \to +\infty$ as $\|\mathbf{w}\| \to \infty$. Because $h \geq 0$, $f = g + h$ is also coercive. This means the sub-level set $\{\mathbf{w} : f(\mathbf{w}) \leq f(\mathbf{0})\}$ is compact, and since f is continuous, it attains its minimum on this set. Strict convexity then rules out two distinct minimizers: if $\mathbf{w}_1 \neq \mathbf{w}_2$ both minimized f, the midpoint $\frac{\mathbf{w}_1+\mathbf{w}_2}{2}$ would give a strictly smaller value, a contradiction. □

1.4.2 Non-smoothness

Unlike the quadratic term, the L_1 penalty is **not differentiable** [10, Slide 4] at any point where some component $w_i = 0$, so f is non-smooth. At points where all $w_i \neq 0$, the gradient exists and is:

$$\nabla f(\mathbf{w}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda_{\text{LASSO}} \operatorname{sign}(\mathbf{w})$$

where $\operatorname{sign}(\mathbf{w})$ is applied element-wise. At points where some $w_i = 0$, one must reason in terms of *subgradients* (see Chapter 2).

1.4.3 Optimality conditions

The optimality condition is expressed via the **subdifferential** [10, Slides 5–6]: $\mathbf{w}^*$ is a global minimizer for the problem if and only if:

$$0 \in \partial f(\mathbf{w}^*) = \mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + \lambda_{\text{LASSO}}\, \partial \|\mathbf{w}^*\|_1$$

where the subdifferential of the L_1 norm is given component-wise by:

$$[\partial \|\mathbf{w}\|_1]_i = \begin{cases} \{+1\} & \text{if } w_i > 0, \\ \{-1\} & \text{if } w_i < 0, \\ [-1, +1] & \text{if } w_i = 0. \end{cases}$$

This condition can be written component-wise as: for each $i = 1, \ldots, H$,

$$\left[\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y})\right]_i + \lambda_{\text{LASSO}} s_i = 0, \qquad s_i \in \partial |w_i^*|. \tag{1.4}$$

Chapter 2

Algorithm 1: Iteratively Reweighted Least Squares

Iteratively Reweighted Least Squares (IRLS) [8] is a classical algorithm for solving optimization problems involving non-smooth objective functions of the form of a p-norm. Its core idea is to handle non-smoothness by solving a sequence of *weighted* least-squares problems, each of which is smooth and admits an efficient closed-form solution. In our setting, IRLS is used to minimize the L_1-regularized least-squares objective:

$$\min_{\mathbf{w}\in\mathbb{R}^H} f(\mathbf{w}) = \underbrace{\frac{1}{2}\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2}_{g(\mathbf{w})} + \underbrace{\lambda_{\text{LASSO}}\|\mathbf{w}\|_1}_{h(\mathbf{w})} \tag{2.1}$$

where $\mathbf{X}\in\mathbb{R}^{M\times H}$, $\mathbf{y}\in\mathbb{R}^M$, and $\lambda_{\text{LASSO}} > 0$ is the regularization hyperparameter of the original problem.

2.1 The core idea: a strict upper bound for the L_1 norm

The difficulty in (2.1) is the L_1 term: convex but non-differentiable at every component crossing zero. To replace it with a smooth surrogate we use the elementary inequality $(|w_i| - |(w_k)_i|)^2 \geq 0$, which after expansion and division by $2|(w_k)_i|$ gives the majorization

$$|w_i| \leq \frac{w_i^2}{2|(w_k)_i|} + \frac{|(w_k)_i|}{2}, \qquad (w_k)_i \neq 0. \tag{2.2}$$

Summing over $i = 1,\ldots,H$ yields the bound on the L_1 norm

$$\|\mathbf{w}\|_1 \leq \frac{1}{2}\mathbf{w}^T\mathbf{W}_k^T\mathbf{W}_k\mathbf{w} + \frac{1}{2}\|\mathbf{w}_k\|_1 \tag{2.3}$$

where $\mathbf{W}_k \in \mathbb{R}^{H\times H}$ is a diagonal matrix with entries:

$$(\mathbf{W}_k)_{ii} = |(w_k)_i|^{-1/2} \tag{2.4}$$

This bounds the non-smooth L_1 term using a *smooth, quadratic* expression $\frac{1}{2}\mathbf{w}^T\mathbf{W}_k^T\mathbf{W}_k\mathbf{w}$, plus a constant term that depends only on the current iterate $\mathbf{w}_k$. Intuitively, $\mathbf{W}_k^T\mathbf{W}_k$ implements a *personalized penalization* of each component of $\mathbf{w}$ [5]: components with small magnitude at iteration k receive a large weight $|(w_k)_i|^{-1}$ and are strongly pushed toward zero, while components with large magnitude receive a small weight and are left relatively free. This *reweighting* mechanism is what drives IRLS to produce sparse iterates, mimicking the sparsity-inducing effect of the L_1 norm through a sequence of smooth quadratic problems.

Handling near-zero components. Equation (2.4) becomes numerically unstable when $(w_k)_i \approx 0$, since $|(w_k)_i|^{-1/2}$ can blow up. To prevent division by (near) zero, a **safety threshold** $\varepsilon_{\text{thr}} > 0$ is introduced:

$$(\mathbf{W}_k)_{ii} = \Big(\max\big(|(w_k)_i|,\, \varepsilon_{\text{thr}}\big)\Big)^{-1/2} \tag{2.5}$$

Typical values are $\varepsilon_{\text{thr}} \in [10^{-6}, 10^{-10}]$.

2.2 The Majorization-Minimization interpretation

IRLS fits naturally inside the Majorization-Minimization (MM) framework [16]. Instead of minimizing f directly, at each iteration k we build a *surrogate* $Q(\mathbf{w}, \mathbf{w}_k)$ that (i) majorizes f, i.e. $Q(\mathbf{w}, \mathbf{w}_k) \geq f(\mathbf{w})$ for all $\mathbf{w}$; (ii) touches f at $\mathbf{w}_k$, i.e. $Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$; and (iii) is smooth, so that it can be minimized in closed form. Plugging the quadratic upper bound (2.3) into (2.1) gives our surrogate,

$$Q(\mathbf{w}, \mathbf{w}_k) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{W}_k\mathbf{w}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1\,, \tag{2.6}$$

which is indeed a quadratic in $\mathbf{w}$. The majorization property is built in via (2.2). The touching property follows from a short computation: at $\mathbf{w} = \mathbf{w}_k$, each summand of the quadratic penalty becomes $(\mathbf{W}_k)_{ii}^2(w_k)_i^2 = (w_k)_i^2/|(w_k)_i| = |(w_k)_i|$, so

$$Q(\mathbf{w}_k, \mathbf{w}_k) = \tfrac{1}{2}\|\mathbf{X}\mathbf{w}_k - \mathbf{y}\|_2^2 + \tfrac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1 + \tfrac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1 = f(\mathbf{w}_k).$$

Taking $\mathbf{w}_{k+1} = \arg\min_{\mathbf{w}} Q(\mathbf{w}, \mathbf{w}_k)$, the two MM properties combined give $f(\mathbf{w}_{k+1}) \leq Q(\mathbf{w}_{k+1}, \mathbf{w}_k) \leq Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$, so the sequence $\{f(\mathbf{w}_k)\}$ is non-increasing. IRLS thus guarantees a monotone decrease of the objective in exact arithmetic; in floating-point arithmetic the monotonicity is preserved up to the accuracy of the linear solver used for the subproblem.

2.3 The weighted least-squares subproblem

To find the next iterate $\mathbf{w}_{k+1}$, we must minimize the surrogate function $Q(\mathbf{w}, \mathbf{w}_k)$ with respect to $\mathbf{w}$:

$$\mathbf{w}_{k+1} = \arg\min_{\mathbf{w}} \left(\frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{W}_k\mathbf{w}\|_2^2 + \frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1 \right)$$

Since the term $\frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{w}_k\|_1$ is constant with respect to $\mathbf{w}$, it does not affect the optimization and can be dropped entirely. The minimization problem simplifies to a standard **weighted least-squares problem with L_2 regularization** (Ridge regression):

$$\mathbf{w}_{k+1} = \arg\min_{\mathbf{w}} \left(\frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda_{\text{IRLS}}\|\mathbf{W}_k\mathbf{w}\|_2^2 \right) \tag{2.7}$$

provided that we strictly define the IRLS regularization parameter as:

$$\lambda_{\text{IRLS}} = \frac{1}{2}\lambda_{\text{LASSO}} \tag{2.8}$$

Relation to the LASSO regularization parameter.
The factor $1/2$ in (2.8) is forced by the majorization constant in (2.3). The upper bound on $\|\mathbf{w}\|_1$ carries a coefficient $\frac{1}{2}$ in front of the quadratic form $\mathbf{w}^T\mathbf{W}_k^T\mathbf{W}_k\mathbf{w}$, so the penalty entering the surrogate (2.6) is $\frac{\lambda_{\text{LASSO}}}{2}\|\mathbf{W}_k\mathbf{w}\|_2^2$. Rewriting this as $\lambda_{\text{IRLS}}\|\mathbf{W}_k\mathbf{w}\|_2^2$ in (2.7) therefore requires $\lambda_{\text{IRLS}} = \frac{1}{2}\lambda_{\text{LASSO}}$. This is precisely the factor that makes the fixed point of IRLS coincide with the LASSO optimum, as shown in the proof of Theorem 2.2.

Proposition 2.1 (Weighted normal equations)**.** *The unique solution of* (2.7) *satisfies:*

$$\left(\mathbf{X}^T\mathbf{X} + 2\lambda_{IRLS}\mathbf{W}_k^T\mathbf{W}_k\right)\mathbf{w}_{k+1} = \mathbf{X}^T\mathbf{y} \tag{2.9}$$

Proof. The gradient of the objective in (2.7) is:

$$\nabla f_k(\mathbf{w}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k\mathbf{w}$$

Setting this to zero yields (2.9). The coefficient matrix $\mathbf{X}^T\mathbf{X} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$ is symmetric positive definite (it is the sum of $\mathbf{X}^T\mathbf{X} \succeq 0$ and $2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k \succ 0$ since $\varepsilon_{\text{thr}} > 0$), so the system has a unique solution. □

Since $\mathbf{W}_k$ is diagonal, defining $\mathbf{Q}_k = \mathbf{X}^T\mathbf{X} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$ and $\mathbf{b} = \mathbf{X}^T\mathbf{y}$, equation (2.9) reduces to:

$$\mathbf{Q}_k\,\mathbf{w}_{k+1} = \mathbf{b} \tag{2.10}$$

This is an $H \times H$ symmetric positive definite (SPD) linear system, which can be solved efficiently at each iteration. Note that $\mathbf{b} = \mathbf{X}^T\mathbf{y}$ can be pre-computed once before the iterative loop, since it does not depend on k. Similarly, $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ is pre-computed once, and only the diagonal $2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$ is updated at each iteration.

Solving the linear system. Because $\mathbf{Q}_k$ is SPD, two efficient methods can be used:

- **Cholesky factorization:** [18] computes $\mathbf{Q}_k = \mathbf{L}\mathbf{L}^T$ in $O(H^3/3)$ operations, then solves two triangular systems in $O(H^2)$. This is the direct method of choice for moderate-sized problems.
- **Conjugate Gradient (CG):** [18] an iterative method that converges in at most H steps in exact arithmetic, with convergence rate depending on the condition number $\kappa(\mathbf{Q}_k)$. Preferable for large-scale problems where H is too large for Cholesky.

2.4 The complete algorithm

Algorithm 1 Iteratively Reweighted Least Squares (IRLS)

Require: $\mathbf{X} \in \mathbb{R}^{M \times H}$, $\mathbf{y} \in \mathbb{R}^M$, $\lambda_{\text{LASSO}} > 0$, $\varepsilon_{\text{thr}} > 0$, $\varepsilon_{\text{stop}} > 0$, $k_{\max}$

Pre-compute $\mathbf{A} \leftarrow \mathbf{X}^T\mathbf{X}$, $\mathbf{b} \leftarrow \mathbf{X}^T\mathbf{y}$
Define $\lambda_{\text{IRLS}} \leftarrow \frac{1}{2}\lambda_{\text{LASSO}}$
Initialize $\mathbf{w}_0 \leftarrow \mathbf{A}^{-1}\mathbf{b}$ (ordinary least-squares solution)
for $k = 0, 1, 2, \ldots, k_{\max}$ **do**
    Compute weights: $(\mathbf{W}_k)_{ii} \leftarrow \big(\max(|(w_k)_i|, \varepsilon_{\text{thr}})\big)^{-1/2}$ for all i
    Assemble system: $\mathbf{Q}_k \leftarrow \mathbf{A} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$
    Solve: $\mathbf{Q}_k\,\mathbf{w}_{k+1} = \mathbf{b}$ *(via Cholesky or CG)*
    if $\|\mathbf{w}_{k+1} - \mathbf{w}_k\| \,/\, \max(1, \|\mathbf{w}_k\|) < \varepsilon_{\text{stop}}$ **then**
        break *(convergence reached)*
    end if
end for
return $\mathbf{w}_{k+1}$

Initialization. We initialize $\mathbf{w}_0$ with the unregularized least-squares solution $\mathbf{A}^{-1}\mathbf{b}$: it is already available (we compute $\mathbf{A}$ and $\mathbf{b}$ once at the start anyway), the relative magnitudes of its components are a reasonable proxy for which weights the L_1 penalty will shrink, and no component is exactly zero, so the weights $(\mathbf{W}_0)_{ii} = |(w_0)_i|^{-1/2}$ are well defined without having to lean on ε_{thr} at the first step.

Stopping criterion. The algorithm terminates when the relative change between successive iterates falls below $\varepsilon_{\text{stop}}$,

$$\frac{\|\mathbf{w}_{k+1} - \mathbf{w}_k\|}{\max(1, \|\mathbf{w}_k\|)} < \varepsilon_{\text{stop}}\,,$$

where the $\max(1, \|\mathbf{w}_k\|)$ normalization makes the test scale-invariant and prevents false triggers when $\|\mathbf{w}_k\|$ is small.

2.5 Convergence analysis

Monotone decrease. As derived in Section 2.2 from the MM framework, IRLS monotonically decreases the objective: $f(\mathbf{w}_{k+1}) \leq f(\mathbf{w}_k)$ for all k. Since f is bounded below by zero, the sequence $\{f(\mathbf{w}_k)\}$ converges.

Theorem 2.2 (Fixed-point consistency)**.** *If the sequence* $\{\mathbf{w}_k\}$ *generated by IRLS converges to a point* $\mathbf{w}^*$ *such that* $|(w^*)_i| > \varepsilon_{\text{thr}}$ *for all* i*, then* $\mathbf{w}^*$ *satisfies the optimality conditions of the original problem* (2.1)*.*

Proof. At a fixed point $\mathbf{w}_k = \mathbf{w}_{k+1} = \mathbf{w}^*$, the weighted normal equations (2.9) give:

$$\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + 2\lambda_{\text{IRLS}}\, \mathbf{W}_*^T\mathbf{W}_*\mathbf{w}^* = \mathbf{0}$$

Since $|(w^*)_i| > \varepsilon_{\text{thr}}$ for all i, the threshold in (2.5) is inactive and $(\mathbf{W}_*^T\mathbf{W}_*)_{ii} = |(w^*)_i|^{-1}$ exactly. The i-th component of the regularizer term is then:

$$(\mathbf{W}_*^T\mathbf{W}_*\mathbf{w}^*)_i = \frac{(w^*)_i}{|(w^*)_i|} = \operatorname{sign}((w^*)_i)$$

Substituting and using $\lambda_{\text{IRLS}} = \frac{1}{2}\lambda_{\text{LASSO}}$ from (2.8):

$$\mathbf{X}^T(\mathbf{X}\mathbf{w}^* - \mathbf{y}) + \lambda_{\text{LASSO}} \operatorname{sign}(\mathbf{w}^*) = \mathbf{0}$$

Since $|(w^*)_i| \neq 0$ for all i, the L_1 norm is differentiable at $\mathbf{w}^*$, so $\partial\|\mathbf{w}^*\|_1 = \{\operatorname{sign}(\mathbf{w}^*)\}$ [10, Slides 5–6]. By the sum rule for subdifferentials [10, Slide 7], the equation above is exactly $\mathbf{0} \in \partial f(\mathbf{w}^*)$, the optimality condition of (2.1) [10, Slides 5–6]. □

Convergence rate. In practice, the IRLS scheme used here exhibits linear (geometric) convergence of the iterates: this is consistent with the analysis carried out by Daubechies et al. [8] for the closely related basis-pursuit IRLS algorithm, where local exponential (linear-rate) convergence is established under sparsity and null-space assumptions on the design matrix. The error $\|\mathbf{w}_k - \mathbf{w}^*\|$ thus shrinks by a constant factor at each iteration, set by the condition number of $\mathbf{Q}_k = \mathbf{X}^T\mathbf{X} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$. The extra diagonal contribution $2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k \succ 0$ behaves like a Tikhonov-style preconditioner, and typically leaves $\mathbf{Q}_k$ better-conditioned than $\mathbf{X}^T\mathbf{X}$ alone, which is what makes the per-iteration linear solve inexpensive and well-conditioned. We do not claim a formal proof of linear convergence for the ELM LASSO setting; the empirical linear rate observed in Section 5.2 is reported as such, with [8] cited only as the closest available analysis under different (basis-pursuit) hypotheses.

Verification of hypotheses for the ELM problem. The two assumptions of Theorem 2.2 are satisfied for (2.1) as follows.

1. *Convergence of the sequence.* We work under the assumption that $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}}\mathbf{W}_1^T)$ has full column rank, which is standard for the ELM setting when $M \geq H$ and $\mathbf{W}_1$ is drawn at random with a nonlinear σ [15].

Under this assumption f is strictly convex (Proposition 1.1) and admits a unique minimizer $\mathbf{w}^*$. The convergence $\mathbf{w}_k \to \mathbf{w}^*$ follows in four steps: *(i)* IRLS monotonically decreases f (Section 2.2) and $f \geq 0$, so $\{f(\mathbf{w}_k)\}$ is non-increasing and bounded below, hence it converges; *(ii)* f is coercive (Proposition 1.1), so every sublevel set is compact and $\{\mathbf{w}_k\}$ has at least one accumulation point $\bar{\mathbf{w}}$; *(iii)* continuity of the surrogate $Q(\cdot, \mathbf{w}_k)$ and the MM descent property $Q(\mathbf{w}_{k+1}, \mathbf{w}_k) \leq Q(\mathbf{w}_k, \mathbf{w}_k) = f(\mathbf{w}_k)$ force $\bar{\mathbf{w}}$ to be a fixed point of the IRLS iteration; *(iv)* by Theorem 2.2, every such fixed point satisfying $|(\bar{w})_i| > \varepsilon_{\text{thr}}$ is a global minimizer, so $\bar{\mathbf{w}} = \mathbf{w}^*$ by uniqueness; since every accumulation point coincides with $\mathbf{w}^*$, the full sequence converges.

2. *Non-zero components at convergence.* The hypothesis $|(w^*)_i| > \varepsilon_{\text{thr}}$ holds for the active (non-zero) components of the sparse solution. Components that collapse to zero within ε_{thr} are effectively frozen by the weight clipping (2.5) and drop out of the optimality condition (1.4) — which is exactly the behaviour we want, because these components correspond to features that the L_1 penalty has marked as inactive.

2.6 Computational cost

The per-iteration cost of IRLS breaks down as follows:

- **Weight update $\mathbf{W}_k$**: $O(H)$ — simply iterating over the components of $\mathbf{w}_k$.
- **Matrix update $\mathbf{Q}_k = \mathbf{A} + 2\lambda_{\text{IRLS}}\mathbf{W}_k^T\mathbf{W}_k$**: $O(H)$ — since $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ is pre-computed and only the diagonal changes.
- **Linear system solve $\mathbf{Q}_k\mathbf{w}_{k+1} = \mathbf{b}$**: $O(H^3/3)$ with Cholesky, or $O(pH^2)$ with CG for p CG steps.

The dominant cost per iteration is the linear system solve: $O(H^3)$ with Cholesky. However, the total number of IRLS iterations is typically small (10–50), and the pre-computation of $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ (costing $O(MH^2)$) is amortized over all iterations. The overall cost is therefore:

$$O(MH^2) + k_{\text{IRLS}} \cdot O(H^3)$$

For problems where $H \ll M$ (many training samples, moderate hidden layer size), the pre-computation dominates and IRLS is very efficient.

2.7 Expected behavior on our problem

From the analysis above we expect the following behaviour on the ELM LASSO problem. The objective $f(\mathbf{w}_k)$ decreases at every iteration, with no oscillations, which makes IRLS easy to monitor and to debug: a single non-decreasing step is

a red flag. On a semilog plot of $f(\mathbf{w}_k) - f^*$ against k, we expect an approximately straight line, with a slope set jointly by the conditioning of $\mathbf{Q}_k$ and by the safety threshold ε_{thr}.

Sparsity is produced dynamically rather than by an external thresholding step: components $(w_k)_i$ that start out small get assigned increasingly large weights $|(w_k)_i|^{-1}$, which pushes them further toward zero at the next iteration; the stable outcome is a weight vector with many components inside an ε_{thr}-ball of zero. The quantitative trade-offs are then driven by two parameters we chose early on: the regularization strength λ_{LASSO} (the model parameter), and the safety threshold ε_{thr} (a purely numerical one). Since $\lambda_{\text{IRLS}} = \frac{1}{2}\lambda_{\text{LASSO}}$ is fixed by the majorization, the only free knob in the model is λ_{LASSO}: larger values give sparser solutions but a higher training residual, smaller values do the opposite; conditioning of $\mathbf{Q}_k$ also depends on λ_{LASSO}, so the convergence speed moves with it. For ε_{thr}, smaller values approximate the true L_1 norm more faithfully and yield sharper sparsity, but pay the price in stability whenever a component drifts very close to zero; a value of order 10^{-8} is a reasonable starting point in this trade-off, and we adopt it as our default to be validated experimentally.

Chapter 3

Algorithm 2: Deflected Subgradient Method

In this chapter, we apply the deflected subgradient method with Polyak step and with target level (SGPTL) to the ELM LASSO problem.

3.1 Motivation: why the plain subgradient method is inadequate for ELM LASSO

The ELM LASSO problem

$$f(\mathbf{w}) = \frac{1}{2}\left\|\mathbf{X}\mathbf{w} - \mathbf{y}\right\|_2^2 + \lambda_{\text{LASSO}}\left\|\mathbf{w}\right\|_1, \tag{3.1}$$

is nonsmooth due to the ℓ_1 penalty. The plain subgradient method converges on convex nonsmooth problems but has two structural limitations on ELM LASSO. First, the update $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i \mathbf{g}_i$ has no memory [10, Slide 10] and zig-zags on ill-conditioned $\mathbf{X}^T\mathbf{X}$; while it attains the optimal iteration complexity $\Omega(L^2/\varepsilon^2)$ [4, Thm. 3.2][1], the constant L depends on $\|\mathbf{X}\|_2$ and the sublevel-set radius. Second, the diminishing-step condition $\sum_i \alpha_i = \infty$, $\sum_i \alpha_i^2 < \infty$ [10, Slide 10] forces fast step shrinkage and stalls practical progress. Deflection injects memory into the search direction and addresses both issues.

3.1.1 The deflection mechanism

Deflected subgradient methods [10, Slide 15] replace the raw subgradient with a direction that blends the current subgradient with the previous search direction:

$$\mathbf{d}_i = \gamma_i\, \mathbf{g}_i + (1 - \gamma_i)\, \mathbf{d}_{i-1} \quad (i \geq 1), \qquad \mathbf{d}_0 = \mathbf{g}_0, \tag{3.2}$$

[1] From the Polyak bound $\bar{f}_k - f^* \leq L\left\|\mathbf{w}_0 - \mathbf{w}^*\right\|/\sqrt{k}$ in [4, Thm. 3.2] one gets $k = O(L^2\|\mathbf{w}_0 - \mathbf{w}^*\|^2/\varepsilon^2)$, which we write as $O(L^2/\varepsilon^2)$ absorbing the initial radius. The form $O(L/\varepsilon^2)$ that appears in some sources is the same order after rescaling ε by L.

where $\gamma_i \in (0, 1]$ is the deflection parameter. The update is $\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i \mathbf{d}_i$; $\gamma_i = 1$ recovers plain subgradient, $\gamma_i < 1$ damps the zig-zag. The method specifies γ_i as the minimiser of $\|\mathbf{d}_i\|^2$ on $[0, 1]$ [10, Slide 15], which maximises the Polyak step α_i since $\|\mathbf{d}_i\|^2$ is its denominator.

3.2 Convergence framework: balancing deflection and stepsize

We use the stepsize-restricted Polyak rule [10, Slide 15]:

$$\alpha_i = \frac{\beta_i \, (f(\mathbf{w}_i) - f^*)}{\|\mathbf{d}_i\|_2^2}, \quad \beta_i \le \gamma_i, \tag{3.3}$$

with $\beta_i = \min(\beta, \gamma_i)$ and $\beta = 1$ [10, Slide 14]. Since f^* is unknown, we substitute the target level $f_{\text{ref}} - \delta$ (Section 3.3).

Deflection floor $\gamma_{\min} > 0$. Condition (3.5) of [7] requires (when $\lambda_k \ge 0$) the uniform bound $\beta_k \ge \beta^* > 0$ on the stepsize multiplier — in our notation $\beta_i \ge \beta^*$ for all i, not just pointwise positivity. Since $\beta_i = \min(1, \gamma_i)$, a uniform $\gamma_i \ge \gamma_{\min} \in (0, 1]$ supplies $\beta_i \ge \gamma_{\min}$, i.e. $\beta^* = \gamma_{\min}$. The closed-form greedy (3.4) gives $\gamma_i \in (0, 1]$ at each step but does not bound the sequence away from zero: when consecutive subgradients align near stationarity, the numerator $\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i, \mathbf{d}_{i-1} \rangle$ of γ^* in (3.5) tends to 0^+ and γ_i decays, dragging β_i and the Polyak step to zero — the iterate freezes. Projecting onto $[\gamma_{\min}, 1]$ with $\gamma_{\min} = 0.05$ supplies the missing uniform bound. A sweep $\gamma_{\min} \in \{0.05, 0.1, 0.2, 0.5\}$ on the convergence instance gives record gaps in $[10^{-7}, 10^{-5}]$, with $\gamma_{\min} = 0.05$ marginally best (Section 5.1).

3.2.1 Optimal deflection parameter

The argmin problem

$$\gamma_i \in \operatorname*{arg\,min}_{\gamma \in [\gamma_{\min}, 1]} \|\gamma \, \mathbf{g}_i + (1 - \gamma) \, \mathbf{d}_{i-1}\|_2^2 \tag{3.4}$$

is quadratic in γ; expanding $\|\gamma \mathbf{g}_i + (1 - \gamma)\mathbf{d}_{i-1}\|^2$ by bilinearity and taking the vertex of the resulting upward parabola gives the closed form (annotated as such in [10, Slide 15]):

$$\gamma^* = \frac{\|\mathbf{d}_{i-1}\|_2^2 - \langle \mathbf{g}_i, \, \mathbf{d}_{i-1} \rangle}{\|\mathbf{g}_i - \mathbf{d}_{i-1}\|_2^2}, \quad \gamma_i = \min(1, \max(\gamma_{\min}, \gamma^*)). \tag{3.5}$$

The clip onto $[\gamma_{\min}, 1]$ instead of $[0, 1]$ is our addition (Section 3.2); the degenerate case $\mathbf{g}_i = \mathbf{d}_{i-1}$ has vanishing leading coefficient and gives $\mathbf{d}_i = \mathbf{d}_{i-1}$ for any γ, so we set $\gamma_i = 1$ by convention. The first iteration is handled analogously:

with $\mathbf{d}_{-1} = \mathbf{0}$ the formula (3.5) returns $\gamma^* = 0$, which would make $\mathbf{d}_0 = \mathbf{0}$; we set $\gamma_0 = 1$ so that $\mathbf{d}_0 = \mathbf{g}_0$, the plain subgradient first step. In the zig-zag regime $\mathbf{g}_i \approx -\mathbf{d}_{i-1}$ with $\|\mathbf{g}_i\| \approx \|\mathbf{d}_{i-1}\|$, the formula gives $\gamma^* \approx 1/2$ and the oscillatory component of $\mathbf{d}_i$ cancels.

3.3 The target-level mechanism

The Polyak stepsize [10, Slide 12] requires f^*. The fundamental inequality [10, Slide 10] is

$$\|\mathbf{w}_{i+1} - \mathbf{w}^*\|_2^2 \leq \|\mathbf{w}_i - \mathbf{w}^*\|_2^2 - 2\alpha_i(f(\mathbf{w}_i) - f^*) + (\alpha_i)^2 \|\mathbf{g}_i\|_2^2, \tag{3.6}$$

minimized at $\alpha_i^* = (f(\mathbf{w}_i) - f^*)/\|\mathbf{g}_i\|_2^2$ [10, Slide 12]. SGPTL replaces f^* with the target level $f_{\text{ref}} - \delta$ [10, Slide 14]: f_{ref} is the best value found so far, $\delta > 0$ an estimate of the residual gap.

A failure mode of this substitution is $f_{\text{ref}} - \delta < f^*$, in which case $f(\mathbf{w}_{i+1}) \geq f^* > f_{\text{ref}} - \delta/2$ for every iterate and the sufficient-descent test $f(\mathbf{w}_{i+1}) \leq f_{\text{ref}} - \delta/2$ is unsatisfiable: f_{ref} never updates and the algorithm silently stalls. The mechanism guards against this through an internal patience counter $r_i = \sum \alpha_j \|\mathbf{d}_j\|$, the displacement accumulated since the last f_{ref} update. When r_i exceeds a threshold R without sufficient descent firing, δ is contracted to $\rho\delta$ with $\rho \in (0, 1)$ and r is reset. Repeated contractions drive $\delta \to 0$, eventually restoring $f_{\text{ref}} - \delta \geq f^*$ and unblocking the sufficient-descent test.

3.4 The complete algorithm

Algorithm 2 presents the full Deflected Subgradient Method with target level for ELM LASSO.

Numerical safeguards. Three pure floating-point guards lie outside the pseudocode. *(i)* If $\|\mathbf{d}_i\|^2 < 10^{-30}$ we terminate. *(ii)* If $f(\mathbf{w}_i) - (f_{\text{ref}} - \delta) \leq 0$ we set $\alpha_i = 0$ (condition (3.5) of [7]) and trigger the sufficient-descent update at $\mathbf{w}_i$, since $f(\mathbf{w}_i) \leq f_{\text{ref}} - \delta/2$. *(iii)* If $\mathbf{w}_{i+1}$ has a non-finite component we skip the iteration without touching $(\delta, f_{\text{ref}}, r)$. No non-theoretical δ-contraction or overshoot rescue is added: such mechanisms have no support in [7] and would break Theorem 3.1.

3.4.1 Parameter calibration for ELM LASSO

SGPTL has five parameters: β, $\gamma_{\min}$, δ_0, ρ, R. None is tuned against f^*.

$\beta = 1$. [7, (3.1)] admits $\beta_k \in (0, 1]$; the value $\beta = 1$ minimises the one-step distance bound [10, Slide 12] and the deflected variant of [10, Slide 15] inherits the property. We fix $\beta = 1$ so that $\beta_i = \min(\beta, \gamma_i)$ is driven by γ_i.

Algorithm 2 Deflected Subgradient with Target Level (SGPTL) for ELM LASSO [10, Slide 15]

Require: f (ELM LASSO objective); $i_{\max}$; $\beta \in (0, 1]$; $\delta_0 > 0$; $R > 0$; $\rho \in (0, 1)$; $\gamma_{\min} \in (0, 1]$

$\mathbf{w}_0 \leftarrow (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ *(same OLS warm start as IRLS)*
$r \leftarrow 0$; $\delta \leftarrow \delta_0$; $f_{\text{ref}} \leftarrow \bar{f} \leftarrow f(\mathbf{w}_0)$
for $i = 0, 1, \ldots, i_{\max}$ **do**
    Compute $\mathbf{g} \in \partial f(\mathbf{w}_i)$ *(subgradient of LASSO)*
    if $i = 0$ **then** *(no $\mathbf{d}_{-1}$ available)*
        $\gamma_i \leftarrow 1$; $\mathbf{d}_i \leftarrow \mathbf{g}$
    else
        Compute γ_i via (3.5) *(optimal deflection, clipped to $[\gamma_{\min}, 1]$)*
        $\mathbf{d}_i \leftarrow \gamma_i\,\mathbf{g} + (1 - \gamma_i)\,\mathbf{d}_{i-1}$ *(deflected direction)*
    end if
    $\beta_i \leftarrow \min(\beta, \gamma_i)$ *(stepsize-restricted, $\beta = 1$ default; see §3.2)*
    $\alpha_i \leftarrow \beta_i\,(f(\mathbf{w}_i) - (f_{\text{ref}} - \delta)) \,/\, \|\mathbf{d}_i\|_2^2$ *(Polyak step)*
    $\mathbf{w}_{i+1} \leftarrow \mathbf{w}_i - \alpha_i\,\mathbf{d}_i$ *(update)*
    $\bar{f} \leftarrow \min(\bar{f},\ f(\mathbf{w}_{i+1}))$ *(update record)*
    if $f(\mathbf{w}_{i+1}) \le f_{\text{ref}} - \delta/2$ **then** *(sufficient descent)*
        $f_{\text{ref}} \leftarrow \bar{f}$; $r \leftarrow 0$
    else if $r > R$ **then** *(travel-distance patience exhausted)*
        $\delta \leftarrow \delta\,\rho$; $r \leftarrow 0$
    else
        $r \leftarrow r + \alpha_i\,\|\mathbf{d}_i\|_2$
    end if
end for
return $\mathbf{w}_{\text{best}}$ (iterate achieving $\bar{f}$)

$\gamma_{\min} = 0.05$**.** Condition (3.5) of [7, p. 365] requires $\beta_k \geq \beta^* > 0$ uniformly; the floor realises this with $\beta^* = \gamma_{\min}$. At $\gamma_{\min} = 0.05$ the clip activates only on degenerate steps and the rate $1/(\gamma_{\min}\sqrt{k})$ of Theorem 3.1 inflates by $20\times$ over the unfloored bound. The sweep $\gamma_{\min} \in \{0.05, 0.1, 0.2, 0.5\}$ in Chapter 5 gives gaps in $[10^{-7}, 10^{-5}]$, $\gamma_{\min} = 0.05$ marginally best.

$\delta_0 = 0.1\, f(\mathbf{w}_0)$**.** [7, Lemma 3.8] requires only $\delta_0 > 0$. We use $f(\mathbf{w}_0)$ (with $\mathbf{w}_0$ the OLS warm start), an f^*-free upper bound on f^*, scaled by $c = 0.1$. The sweep $c \in \{0.01, 0.05, 0.1, 0.5, 1\}$ in Section 5.3 produces gaps within one order of magnitude, and the diagnostic oracle $0.1\, f^*$ is numerically indistinguishable.

$\rho = 0.7$**.** [7, Lemma 3.8] requires $\mu \in (0, 1)$. Sweeping $\rho \in \{0.3, 0.5, 0.7, 0.8, 0.9, 0.95\}$ on the synthetic and real-data instances, $\rho = 0.7$ is at most a factor of six away from the per-instance best.

$R = 1$**.** [7, Lemma 3.8] requires $R > 0$. On ELM LASSO with the OLS warm start, $\alpha_i \|\mathbf{d}_i\| \sim 10^{-3}$, so $R = 1$ fires the contraction every $\sim 10^3$ stalled iterations. The first submission used $R \approx 894$, three orders of magnitude above this scale, so the $r > R$ branch never fired (see Section 5.1.1).

3.4.2 Starting point and patience-threshold calibration

Algorithm 2 uses the same OLS warm start as IRLS, $\mathbf{w}_0 = (\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top \mathbf{y}$, for an apples-to-apples comparison. With $R = 1$ and $\gamma_{\min} = 0.05$ the travel-distance branch fires $\mathcal{O}(10)$–$\mathcal{O}(100)$ times per 8 000-iteration run, contracting δ at the pace the theory expects (Section 5.3).

3.5 Application to the ELM LASSO problem

3.5.1 Subgradient computation for ELM LASSO

For the ELM LASSO objective, decompose $f = g + h$ where $g(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$ (smooth) and $h(\mathbf{w}) = \lambda_{\text{LASSO}} \|\mathbf{w}\|_1$ (nonsmooth).

By the sum rule for subdifferentials [10, Slide 7]:

$$\partial f(\mathbf{w}) = \nabla g(\mathbf{w}) + \lambda_{\text{LASSO}}\, \partial \|\mathbf{w}\|_1 ,$$

where $\nabla g(\mathbf{w}) = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$ and the ℓ_1 subdifferential is component-wise (Section 1.4.3):

$$\mathbf{g} = \mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda_{\text{LASSO}}\, \mathbf{s}, \quad s_i \in \partial|w_i|. \tag{3.7}$$

For $w_i = 0$ we take $s_i = 0$ via `numpy.sign(0)`, which is admissible (it lies in $[-1, 1]$) so Theorem 3.1 applies. It is not the minimum-norm element; the discrepancy in $|g_i|$ is at most λ_{LASSO} per component, inflating L but not the $O(L^2/\varepsilon^2)$ order. The issue is moot under Polyak-step updates from the OLS warm start: exact zeros occur on a set of Lebesgue measure zero.

3.5.2 Convergence analysis and hypothesis verification for ELM LASSO

Theorem 3.1 (Convergence of SGPTL, adapted from [10, Slides 14–15], [7])**.** *Let $f : \mathbb{R}^H \to \mathbb{R}$ be convex with $f^* = \min_{\mathbf{w}} f(\mathbf{w}) > -\infty$ attained at some $\mathbf{w}^*$, and let $\{\mathbf{w}_i\}$ be the sequence generated by Algorithm 2 (the target-level deflected subgradient method we actually run). Assume the subgradients encountered along the run are uniformly bounded, $\|\mathbf{g}_i\|_2 \le L$, and the stepsize-restricted rule $\beta_i \le \gamma_i$ holds with $\gamma_i \ge \gamma_{\min} > 0$. Then the record values $\bar{f}_i = \min_{h \le i} f(\mathbf{w}_h)$ converge to f^*, and once the target-level estimate has sharpened to f^* (i.e. $\delta_k \to 0$ and $f^k_{\mathrm{ref}} \to f^*$, which Lemma 3.8 of [7] guarantees) the asymptotic rate matches the classical Polyak bound [4, Thm. 3.2], [10, Slide 13] inflated by the deflection floor:*

$$\bar{f}_k - f^* \;\le\; \frac{L\,\|\mathbf{w}_0 - \mathbf{w}^*\|_2}{\gamma_{\min}\sqrt{k}}\,, \tag{3.8}$$

so the worst-case complexity to attain $\bar{f}_k - f^* \le \varepsilon$ is $O(L^2/(\gamma_{\min}^2 \varepsilon^2))$.

Proof. We verify the hypotheses of [7, Theorem 3.7] on Algorithm 2 and read off the conclusions. The notation in [7] maps to ours via $\sigma_k \leftrightarrow 0$ (exact oracle), $\alpha_k \leftrightarrow \gamma_i$, $\beta_k \leftrightarrow \beta_i$, $\nu_k \leftrightarrow \alpha_i$, $\mu \leftrightarrow \rho$, $X = \mathbb{R}^H$ (no constraint), and $\lambda_k \leftrightarrow f(\mathbf{w}_i) - (f^k_{\mathrm{ref}} - \delta_k)$.

(a) Deflection-consistency condition (2.13). In [7] condition (2.13) of Lemma 2.4 reads $d_k = \tilde{d}_k \Rightarrow v_{k+1} = \tilde{d}_k$, where $\tilde{d}_k$ is the unprojected deflected direction and $\hat{d}_k = -P_{T_k}(-\tilde{d}_k)$ the projected one. This is the hypothesis under which the "crucial" inequality (2.12) holds, which the stepsize-restricted analysis relies on. With $X = \mathbb{R}^H$ the tangent cone is $T_k = \mathbb{R}^H$ everywhere, so $\hat{d}_k = \tilde{d}_k$ and necessarily $d_k = \tilde{d}_k$ at every iteration; the algorithm sets v_{k+1} to $d_k = \tilde{d}_k$ (Algorithm 2 carries no projection step), so (2.13) is satisfied trivially.

(b) Stepsize condition (3.5): $\lambda_k \ge 0 \Rightarrow \alpha_k \ge \beta_k \ge \beta^* > 0$, $\lambda_k < 0 \Rightarrow \alpha_k = 0$. When $\lambda_k > 0$, line 12 of Algorithm 2 sets $\beta_i = \min(1, \gamma_i) \ge \gamma_{\min}$ via the floor on γ_i, so we may take $\beta^* = \gamma_{\min}$; the inequality $\alpha_k \ge \beta_k$ in (3.5) is $\gamma_i \ge \beta_i$, which is exactly the stepsize-restricted rule. When $\lambda_k \le 0$ the numerical safeguard of Section 3.4 skips the step — equivalent to $\alpha_i = 0$ — and triggers the sufficient-descent update $f^k_{\mathrm{ref}} \leftarrow \bar{f}$, $r \leftarrow 0$ on $\mathbf{w}_i$ itself. The skip realises (3.5) verbatim; the additional f^k_{ref} update is theory-aligned because $\lambda_k \le 0$ means $f(\mathbf{w}_i) \le f^k_{\mathrm{ref}} - \delta_k \le f^k_{\mathrm{ref}} - \delta_k/2$, which is the sufficient-descent condition of [7, Lemma 3.8] evaluated at $\mathbf{w}_i$ rather than at $\mathbf{w}_{i+1}$.

(c) Target-level threshold condition (3.26): requires *either* $f^\infty_{\mathrm{ref}} = f^* = -\infty$ *or* $\liminf_k \delta_k = 0$ together with the series condition (3.25), $\sum_k \lambda_k / \|\mathbf{d}_k\|^2 = \infty$ (equivalent to $\sum_k s_k = \infty$ via (3.28) of [7], where $s_k = \|\mathbf{w}_{k+1} - \mathbf{w}_k\| = \beta_k \lambda_k / \|\mathbf{d}_k\|$). Since on ELM LASSO $f^* > -\infty$, the first branch is excluded and we need the second. [7, Lemma 3.8] provides a concrete update strategy for $(f^k_{\mathrm{ref}}, \delta_k)$ — the sufficient-descent test on $f_{k+1} \le f^k_{\mathrm{ref}} - \delta_k/2$, the target-infeasibility test on $r_k > R$, and the accumulate-r counter — which attains both $\delta_k \to 0$ and $\sum_k s_k = \infty$. Lines 16–21 of Algorithm 2 implement that strategy verbatim under the identification $\mu \leftrightarrow \rho$, so condition (3.26) holds.

Subgradients uniformly bounded. The convex objective f of ELM LASSO is coercive and continuous; the iterates $\{\mathbf{w}_i\}$ remain in a compact sublevel set S_{c_0} for some $c_0 \geq f(\mathbf{w}_0)$ (verification below), and ∂f is uniformly bounded on S_{c_0} by Lipschitz continuity of convex functions on compact sets [11, Slide 9].

Conditions (2.13), (3.5), (3.26) being verified with $\sigma^* = 0$, [7, Theorem 3.7] gives $f_{\text{ref}}^{\infty} \leq f^* + \sigma^* = f^*$, so $\bar{f}_i \to f^*$. The asymptotic rate (3.8) is then the classical Polyak rate [4, Thm. 3.2], [10, Slide 13] applied to the regime where the target-level estimate has sharpened to f^* (Lemma 3.8 of [7] guarantees $\delta_k \to 0$, so eventually the step reduces to the standard Polyak step with f^*); the factor $1/\gamma_{\min}$ is the explicit price of the deflection floor, which contracts the effective stepsize by $\beta_i = \min(1, \gamma_i) \geq \gamma_{\min}$ at every iteration. On ELM LASSO with $\gamma_{\min} = 0.05$ this inflates the iteration count to reach a given ε by $400\times$ relative to the unfloored Polyak rate, but the $O(L^2/\varepsilon^2)$ order is preserved. □

Verification of theorem assumptions for ELM LASSO.

1. **Convexity.** $\nabla^2 g = \mathbf{X}^T\mathbf{X} \succeq 0$, the ℓ_1 norm is convex, so f is convex [12, Slides 21–24]. Under the standard ELM assumption $M \geq H$ with $\mathbf{X}$ full column rank [15], $\nabla^2 g \succ 0$ and f is μ-strongly convex with $\mu = \lambda_{\min}(\mathbf{X}^T\mathbf{X}) > 0$ [11, Slide 12], [12, Slides 21–24]. SGPTL exploits neither acceleration nor proximal structure, so its guarantee remains $O(L^2/\varepsilon^2)$.

2. **Bounded subgradients.** The ℓ_1 part is bounded, $\|\lambda_{\text{LASSO}}\mathbf{s}\|_2 \leq \lambda_{\text{LASSO}}\sqrt{H}$. The smooth part $\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$ grows with $\|\mathbf{w}\|$, so f is not globally Lipschitz. But f is coercive, every sublevel set is compact, and the target-level mechanism keeps $f(\mathbf{w}_i) - (f_{\text{ref}} - \delta)$ on the same scale as $f(\mathbf{w}_i) - f^*$, so the iterates stay in an enlarged sublevel set S_{c_0} with $c_0 \geq f(\mathbf{w}_0)$. Lipschitz continuity of f on S_{c_0} [11, Slide 9] gives the uniform bound L.

Complexity. The lower bound for first-order methods on nonsmooth convex problems is $\Omega(L^2/\varepsilon^2)$ [10, Slide 8], [4]; Algorithm 2 attains it. Deflection improves constants, not the order.

3.6 Expected practical behavior on the ELM LASSO problem

SGPTL is non-monotone: only $\bar{f}_i = \min_{h \leq i} f(\mathbf{w}_h)$ decreases to f^*, so semilog plots use $\log(\bar{f}_i - f^*)$ and the returned iterate is $\mathbf{w}_{\text{best}}$. The rate is sublinear $O(L^2/\varepsilon^2)$, so accuracies below 10^{-6} are unrealistic. The per-iteration cost is $O(MH)$ (two matvecs with $\mathbf{X}$) against $O(H^3)$ for IRLS's Cholesky solve; DSM is CPU-competitive only at large H and moderate accuracy.

No reliable subgradient stopping test exists: $0 \in \partial f(\mathbf{w}^*)$ does not force $\|\mathbf{g}_i\| \to 0$, so we stop on $i_{\max}$ or on a flat-window test on $\bar{f}_i$. Neither method produces exact zeros: IRLS pins them at the safety floor $\varepsilon_{\text{thr}} = 10^{-8}$ in (2.5),

SGPTL leaves them at the residual-rate level $\sim L/\sqrt{i}$ (10^{-3}–10^{-4} at our budgets). The fair comparison uses the same post-termination threshold on both (Section 5.4). Larger λ_{LASSO} amplifies oscillations through $\mathbf{s}$ and inflates the iteration count.

Chapter 4

Algorithm Comparison

Both IRLS (Chapter 2) and the Deflected Subgradient Method (Chapter 3) solve the same LASSO problem

$$\min_{\mathbf{w}\in\mathbb{R}^H} f(\mathbf{w}) = \frac{1}{2}\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2 + \lambda_{\text{LASSO}}\|\mathbf{w}\|_1, \tag{4.1}$$

but handle the non-smoothness of the ℓ_1 penalty in opposite ways.

4.1 Core strategy: how each algorithm handles non-smoothness

IRLS — smoothing via majorization.
IRLS replaces f with a smooth quadratic majorant $Q(\mathbf{w}, \mathbf{w}_k)$ tight at $\mathbf{w}_k$ (Section 2.2), reducing each step to a weighted ridge problem solved in closed form via the normal equations (Section 2.3). The ℓ_1 penalty is absorbed into diagonal weights $(\mathbf{W}_k)_{ii}$ that blow up where $|w_{k,i}|$ is small, driving those components to zero (Section 2.1).

DSM — direct subgradient with memory.
DSM descends along a subgradient $\mathbf{g} \in \partial f(\mathbf{w}_i)$ (Section 3.5.1), deflected with the previous direction (Section 3.1.1) to damp zig-zag, with a Polyak step using a self-adapting target estimate of f^* (Section 3.3).

IRLS solves a sequence of smooth problems *exactly* and is monotone in f; DSM operates on f directly, accepts $f(\mathbf{w}_i)$ may increase, and tracks progress via the record value $\bar{f}^{\,i}$.

4.2 Convergence behaviour

4.2.1 Monotonicity

MM guarantees IRLS is **monotone**: $f(\mathbf{w}_{k+1}) \leq f(\mathbf{w}_k)$ (Section 2.2); a non-decreasing plot signals a bug, and an iterate-change criterion reliably stops the algorithm. DSM is **non-monotone**: only the record $\bar{f}^i = \min_{h \leq i} f(\mathbf{w}_h)$ is monotone, so DSM returns $\mathbf{w}_{\text{best}}$ and stops on a fixed budget.

4.2.2 Rate

IRLS achieves a **linear** rate (Section 2.5); DSM achieves the optimal **sublinear** $O(1/\varepsilon^2)$ rate for nonsmooth convex functions (Theorem 3.1). Tightening ε by one decade costs $\sim 100\times$ more DSM iterations versus a constant for IRLS.

4.2.3 Convergence on the ELM problem

For the ELM problem $\mathbf{X} = \sigma(\mathbf{X}_{\text{origin}}\mathbf{W}_1^T)$ has full column rank (standard ELM condition when $M \geq H$ with random $\mathbf{W}_1$ and nonlinear σ), so f is μ-strongly convex with unique minimizer $\mathbf{w}^*$. IRLS iterates $\mathbf{w}_k \to \mathbf{w}^*$ at a linear rate governed by the weighted normal-equation conditioning (Theorem 2.2); Theorem 3.1 only guarantees $\bar{f}_i \to f^*$, with iterates that may oscillate. Strong convexity would allow $O(L^2/(\mu\varepsilon))$ [11, Slide 12], but SGPTL does not exploit it (Section 3.5.2).

4.3 Computational cost

4.3.1 Per-iteration cost

Operation	**IRLS**	**DSM**
Weight / direction update	$O(H)$	$O(H)$
Matrix-vector product(s)	—	$O(MH)$
Linear system solve	$O(H^3)$ (Cholesky) or $O(pH^2)$ (CG)	—
Total per iteration	$O(H^3)$	$O(MH)$

Table 4.1: Per-iteration computational cost of IRLS and DSM.

IRLS solves an $H \times H$ SPD system per iteration (Section 2.3): $O(H^3/3)$ via Cholesky, or $O(pH^2)$ via p CG steps when H is large and $\mathbf{Q}_k$ well-conditioned. The matrices $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ and $\mathbf{b} = \mathbf{X}^T\mathbf{y}$ are precomputed once at $O(MH^2)$ and $O(MH)$.

DSM uses two matrix-vector products per iteration ($\mathbf{X}\mathbf{w}_i$ and $\mathbf{X}^T(\cdot)$, each $O(MH)$), no precomputation (Section 3.5.1).

4.3.2 Total cost

With iteration counts k_{IRLS}, k_{DSM}:

$$\begin{aligned} \text{IRLS total:} &\quad O(MH^2) + k_{\text{IRLS}} \cdot O(H^3), \\ \text{DSM total:} &\quad k_{\text{DSM}} \cdot O(MH). \end{aligned}$$

Linear vs sublinear gives $k_{\text{IRLS}} \ll k_{\text{DSM}}$, so DSM is competitive only when $H^2 \gg M$ and the Cholesky cost dominates.

On the ELM problem $M \gg H$ is typical, so $O(MH^2)$ precomputation dominates IRLS's total. Total costs become $O(MH^2)$ vs $O(k_{\text{DSM}} \cdot MH)$, and IRLS wins whenever $k_{\text{DSM}} \gtrsim H$. Since Theorem 3.1 forces $k_{\text{DSM}} = O(L^2/\varepsilon^2)$ with L growing in H (the ℓ_1 subgradient has norm $\leq \lambda_{\text{LASSO}}\sqrt{H}$), this inequality holds at every H we test for any reasonable accuracy.

4.4 Solution quality

4.4.1 Sparsity

IRLS yields numerically sparse iterates as a byproduct of its weighted least-squares structure: small components accumulate large weights and are driven to exactly zero (within ε_{thr}) at convergence, no post-processing (Section 2.7). DSM gives only **approximate sparsity**; an explicit thresholding step is needed (Section 3.6).

4.4.2 Accuracy

At equal budget IRLS reaches much higher accuracy on small-to-moderate problems thanks to its linear rate. DSM is competitive only when $H^2 \gg M$ (Table 4.1).

4.5 Comparison summary for the ELM problem

On the project's regime ($M \gg H$, $\mathbf{W}_1$ fixed, offline training) IRLS is preferred: the $O(MH^2)$ precomputation of $\mathbf{A}$ amortizes over a few cheap Cholesky solves, sparsity comes for free, the single knob is ε_{thr}, and monotone f doubles as a correctness check. DSM is the better choice only when $H^2 \gg M$ or $\mathbf{X}^T\mathbf{X}$ does not fit in memory, and the accuracy target is moderate ($\varepsilon \sim 10^{-3}$).

Criterion	**IRLS**	**DSM**
Approach	Smoothing (MM)	Direct subgradient
Monotone	Yes	No (record value only)
Convergence rate	Linear	Sublinear $O(1/\varepsilon^2)$
Per-iteration cost	$O(H^3)$	$O(MH)$
Precomputation	$O(MH^2)$	None
Exact sparsity	Yes	No (needs thresholding)
Parameters to tune	ε_{thr}	δ_0, ρ, R, β
Stopping criterion	Reliable (iterate change)	Unreliable (fixed budget)
Preferred when	High accuracy, $H \ll M$	Large H, moderate accuracy

Table 4.2: Summary comparison of IRLS and DSM on the ELM LASSO problem.

Chapter 5

Experimental Results

This chapter reports experiments on the IRLS and SGPTL implementations of Chapters 2 and 3, against the predictions of Chapter 4: linear rate with natural sparsity for IRLS, sublinear non-monotone progress with approximate sparsity for SGPTL, and the $O(MH^2) + k_{\text{IRLS}} \cdot O(H^3)$ versus $k_{\text{SGPTL}} \cdot O(MH)$ cost trade-off.

5.1 Experimental setup

Implementation. The codebase under `progetto/code/src/` is organised in five modules: `lasso_utils.py` (shared objective, smooth-part gradient, soft-threshold subgradient selection), `linear_solvers.py` (Cholesky via `scipy.linalg.cho_factor/cho_solve` and a hand-rolled Jacobi-preconditioned CG), `irls.py` (Algorithm A1 driver), `deflected_subgradient.py` (Algorithm A2 with deflection floor $\gamma_{\min}$, clip $\beta_i = \min(\beta, \gamma_i)$, Polyak target-level), and `elm.py` (random hidden layer plus activations). Core algorithms are implemented from scratch; only NumPy/SciPy primitives are reused. A pytest suite under `progetto/code/tests/` cross-checks both algorithms against `sklearn.linear_model.Lasso` and verifies surrogate-descent and record-monotonicity invariants iterate-by-iterate.

Reference solution. We compute the reference $\mathbf{w}^*$ via `sklearn.linear_model.Lasso` (coordinate descent, `tol`$= 10^{-12}$, 10^5 iterations, `fit_intercept=False`). Sklearn minimises

$$f_{\text{sklearn}}(\mathbf{w}) \;=\; \frac{1}{2M}\,\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2 \;+\; \alpha_{\text{sklearn}}\,\|\mathbf{w}\|_1\,, \tag{5.1}$$

which shares an arg-min with our f in eq. (1.3) when $\alpha_{\text{sklearn}} = \lambda_{\text{LASSO}}/M$. We then evaluate $f^* = f(\mathbf{w}^*)$ directly from eq. (1.3). The returned $\mathbf{w}^*$ satisfies the KKT condition (1.4) to below 10^{-3} on every instance.

Synthetic data. For every experiment we generate the data the same way. We sample a sparse ground-truth weight vector $\mathbf{w}_{\text{true}} \in \mathbb{R}^H$ with 10% to 15%

non-zero entries drawn from $\mathcal{N}(0,1)$, build a random feature matrix $\mathbf{X} \in \mathbb{R}^{M \times H}$ with i.i.d. Gaussian columns normalised to unit ℓ_2 norm, and set $\mathbf{y} = \mathbf{X}\,\mathbf{w}_{\text{true}} + \boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, 0.05^2\,\mathbf{I})$.

Why Gaussian. Column-normalised Gaussian designs are mutually incoherent and keep $\kappa(\mathbf{X}^\top\mathbf{X})$ moderate when $M \geq 2H$, the benign regime in which our analysis applies [14]. The SNR $\|\boldsymbol{\varepsilon}\|_2 \,/\, \|\mathbf{X}\mathbf{w}_{\text{true}}\|_2$ stays in the 5–10% range across the sizes tested.

ELM extension. For the full pipeline, $\mathbf{X}$ is the matrix of hidden activations $\sigma(\mathbf{X}_{\text{origin}}\,\mathbf{W}_1^\top)$ with sigmoid σ, fixed random $\mathbf{W}_1$, Gaussian $\mathbf{X}_{\text{origin}}$. The sigmoid breaks the unit-norm column property, so the real-data experiments of Section 5.7 report $\text{cond}(\mathbf{X}^\top\mathbf{X})$ explicitly.

Timing protocol. Wall-clock times are taken with `time.perf_counter` and exclude the data-generation step. They reflect single-thread Python execution without explicit BLAS-thread tuning.

The deflection floor in practice. Section 3.2 motivates the lower bound $\gamma_i \geq \gamma_{\min}$ as a structural ingredient for the clip $\beta_i = \min(\beta, \gamma_i)$. Empirically, on the convergence instance ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $i_{\max} = 4000$) the unfloored variant ($\gamma_{\min} = 0$) spends 97% of iterations in the numerator-skip branch, δ underflows and the record gap stalls at $3.9 \cdot 10^{-2}$; a full (δ_0, ρ) sweep does not recover the loss. With $\gamma_{\min} = 0.05$ the gap descends to $3 \cdot 10^{-5}$ in the same budget. The sweep $\gamma_{\min} \in \{0.05, 0.1, 0.2, 0.5\}$ keeps the final gap in 10^{-7}–10^{-5}, with $\gamma_{\min} = 0.05$ marginally best (Figure 5.1).

5.1.1 Implementation note: from the first submission to the theory-pure rule

The first submission carried, on top of Algorithm 2, an iteration-count δ-contraction fallback, an overshoot rescue snapping $\mathbf{w}$ back to $\mathbf{w}_{\text{best}}$, and the default $R = 10\sqrt{i_{\max}} \approx 894$ at $i_{\max} = 8000$ — far above any realistic per-iteration step length ($\alpha_i \,\|\mathbf{d}_i\| \sim 10^{-3}$), so the $r > R$ branch never fired and the iteration-count fallback drove every δ-contraction. The instructor flagged both safeguards as inconsistent with [7, Lemma 3.8], which requires every contraction to follow $r_k > R$ travel. This submission removes both workarounds and sets $R = 1$ to match the natural step length; on the convergence instance this gives ~ 22 contractions over 8000 iterations at $\rho = 0.7$ (against ~ 95 under the old fallback and 0 under the old R). $R = 1$ is the one calibration outside the pseudocode; the theory requires only $R > 0$. The new gap on the convergence instance is $4.8 \cdot 10^{-5}$ against the first submission's $\sim 6.6 \cdot 10^{-8}$; the previous figure was a numerical artefact of the iteration-count fallback. The qualitative comparison (IRLS faster, SGPTL cheaper per iteration) is preserved. First-submission artefacts are retained under `progetto/code/results_old_submission/`.

Warm vs cold start for SGPTL. Under the old R cold start dominated (warm start triggered blow-ups); under $R = 1$ the two are essentially neutral. Figure 5.1 shows both starts on the convergence instance: the cold start ends at $2.4 \cdot 10^{-5}$ with 26 δ-contractions, the warm start at $4.8 \cdot 10^{-5}$ with 22. Across similar instances neither is uniformly better. We keep the OLS warm start as the default for SGPTL because it is the same starting point used by IRLS, which makes the comparison apples-to-apples.

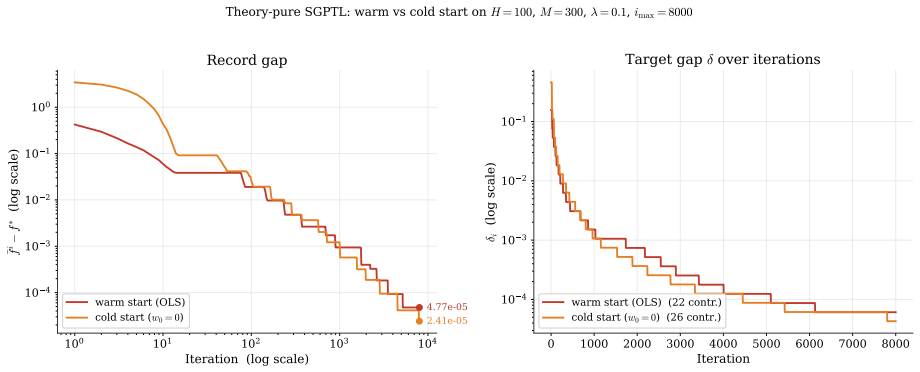

Figure 5.1: Theory-pure SGPTL with the OLS warm start versus cold start ($\mathbf{w}_0 = \mathbf{0}$) on the same synthetic instance ($H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, $i_{\max} = 8000$, $\rho = 0.7$, $\gamma_{\min} = 0.05$, $R = 1$). Left panel: record gap $\bar{f}^i - f^*$ on log–log axes; both starts produce sublinear descent. Right panel: δ_i staircase (26 contractions cold, 22 warm). Final gaps within a factor of two; warm start is the default for consistency with IRLS.

5.2 Convergence behaviour

We start from the convergence experiment with $H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$, sparsity 10%. Both algorithms use the OLS warm start $\mathbf{w}_0 = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ (Section 3.4).

Cost of the warm start. The OLS warm start solves $\mathbf{X}^\top \mathbf{X}\, \mathbf{w}_0 = \mathbf{X}^\top \mathbf{y}$ via Cholesky at cost $O(MH^2) + O(H^3/3)$ — one IRLS-iteration of work on the convergence instance, at most $\sim 40\%$ of the SGPTL budget on the largest scalability instance, well below 1% elsewhere. Both algorithms pay it once and we include it in their wall-clock totals. On the convergence instance $f(\mathbf{0}) \approx 76$, $f(\mathbf{w}_0) \approx 1.56$, $f^* \approx 1.13$: the warm start lands $\sim 180\times$ closer to the optimum than $\mathbf{0}$, a gap that SGPTL would need thousands of iterations to recover under the $O(L^2/\varepsilon^2)$ rate. Using the same $\mathbf{w}_0$ for both algorithms also removes start as a confounder in the comparison.

Figure 5.2 shows the gap $f(\mathbf{w}_k) - f^*$ versus iteration. IRLS produces a straight line on the semilog left panel — the linear MM rate of Section 2.5 —

reaching $1.5 \cdot 10^{-6}$ in 100 iterations, with iterate change $5.6 \cdot 10^{-6}$.

SGPTL (8000 iterations, $\beta = 1$, $\gamma_{\min} = 0.05$, $\delta_0 = 0.1\, f(\mathbf{w}_0)$, $\rho = 0.7$) gives the right panel: the current gap oscillates as expected, the record descends along the sublinear envelope to a final $4.8 \cdot 10^{-5}$. Each additional decade multiplies the iteration count by $\sim$100, so the gap floor is budget-limited.

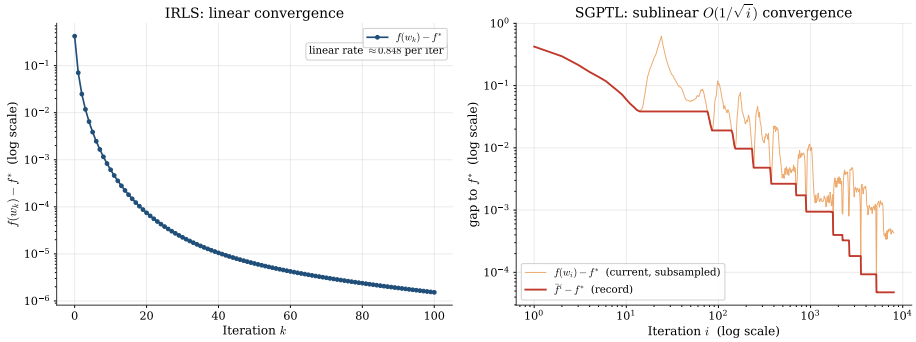

Figure 5.2: Optimality gap versus iteration count on a problem with $H = 100$, $M = 300$, $\lambda_{\text{LASSO}} = 0.1$. Left: IRLS on a semilog axis; the in-panel rate annotation is the empirical per-iteration multiplicative reduction sampled in the linear region. Right: SGPTL on log–log axes, with the current value $f(\mathbf{w}_i) - f^*$ (orange, oscillating; subsampled log-uniformly so the late-iteration density does not collapse into a solid band) overlaid on the record $\bar{f}^i - f^*$ (red). The record traces the sublinear $O(1/\sqrt{i})$ envelope predicted by Theorem 3.1: gaining each decade of accuracy multiplies the iteration count by $\sim$100.

Figure 5.3 replots against wall-clock time. On this size SGPTL's per-iteration advantage ($O(MH)$ vs $O(H^3)$) does not compensate for the slower rate: IRLS reaches 10^{-6} in ~ 3 ms, SGPTL falls below 10^{-2} within ~ 10 ms and then converges sublinearly to $\sim 5 \cdot 10^{-5}$ over the remaining $\sim$150 ms.

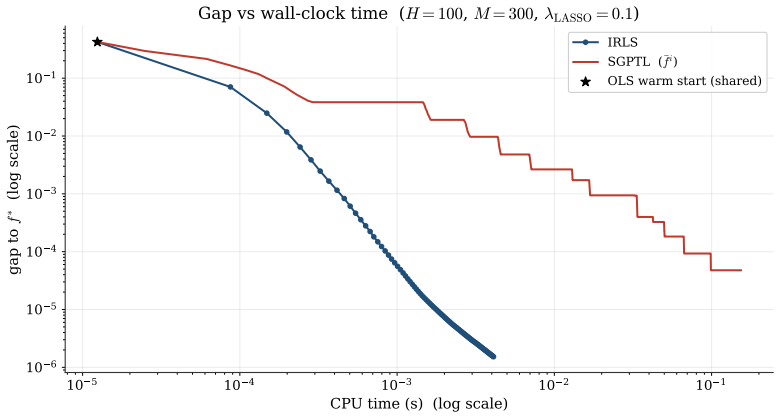

Figure 5.3: Optimality gap versus CPU time on the same instance as Figure 5.2, log–log axes.

Non-monotonicity (Section 3.3) is visible empirically in Figure 5.4.

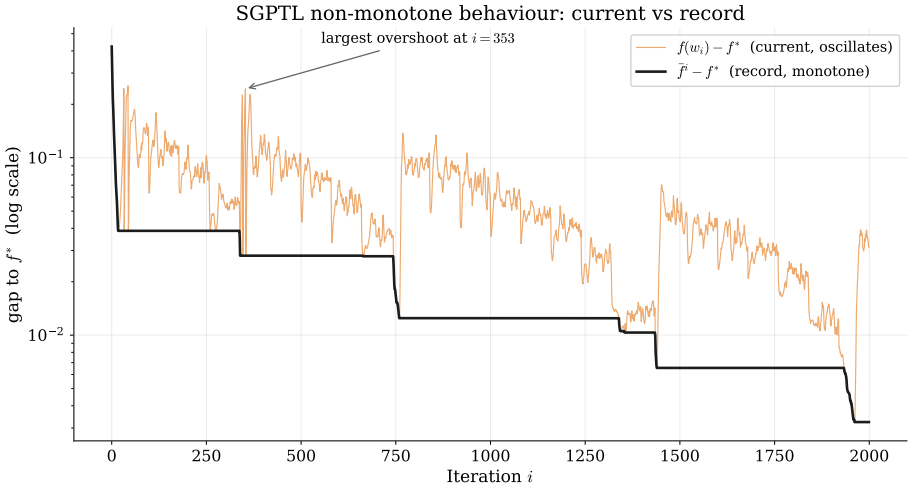

Figure 5.4: SGPTL on the convergence instance, first 2000 iterations, semilog y axis. The current gap $f(\mathbf{w}_i) - f^*$ (orange) spikes well above the record gap $\bar{f}^i - f^*$ (black) at several points throughout the run, while the record is monotone non- increasing by construction. The largest overshoot is annotated. This is the practical reason why the algorithm returns $\mathbf{w}_{\text{best}}$ rather than the last iterate.

5.3 Parameter sensitivity

5.3.1 IRLS: ε_{thr} and λ_{LASSO}

The two parameters acting on IRLS are the safety threshold ε_{thr} in (2.5) and λ_{LASSO}. We sweep both at $H = 100$, $M = 400$, 100 iterations.

Figure 5.5 (left): for $\varepsilon_{\text{thr}} \in \{10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}, 10^{-12}\}$ the final gap tracks ε_{thr} ($1.4 \cdot 10^{-6}$, $1.5 \cdot 10^{-8}$, $5.3 \cdot 10^{-10}$ for $10^{-6}, 10^{-8}, 10^{-10}$); 10^{-4} is too loose ($1.4 \cdot 10^{-4}$, no sparsity); 10^{-12} hits floating-point limits. Sparsity stabilises at 86% (reference 88%). The default 10^{-8}–10^{-10} used elsewhere is the sweet spot.

Right: $\lambda_{\text{LASSO}} \in \{0.01, 0.05, 0.1, 0.5, 1.0\}$. Convergence speed is roughly invariant (λ enters $\mathbf{Q}_k$ as a bounded conditioning factor); sparsity moves from 11% to 95%, monotone in λ as predicted by (1.4).

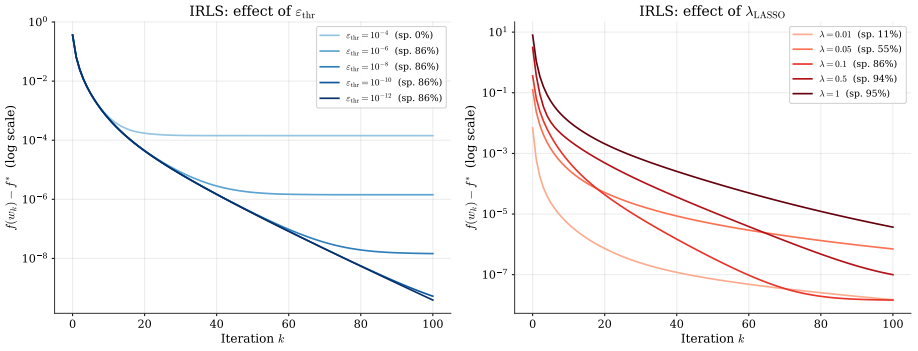

Figure 5.5: IRLS sensitivity. Left: effect of ε_{thr} on convergence and sparsity at $\lambda_{\text{LASSO}} = 0.1$. Right: effect of λ_{LASSO} at $\varepsilon_{\text{thr}} = 10^{-8}$.

5.3.2 IRLS: linear solver choice (Cholesky vs. Conjugate Gradient)

Chapter 2 identifies Cholesky ($O(H^3/3)$) and CG ($O(pH^2)$ per CG step) as candidate inner solvers for $\mathbf{Q}_k \mathbf{w}_{k+1} = \mathbf{b}$; the trade-off depends on H and $\kappa(\mathbf{Q}_k)$. We compare them on the instance $H = 100$, $M = 400$, $\lambda_{\text{LASSO}} = 0.1$, OLS warm start, $\varepsilon_{\text{thr}} = 10^{-8}$, $k_{\max} = 300$ (Table 5.1).

Solver	Total time (ms)	Iterations	Sparsity at 10^{-6}	Converged
Cholesky	11.7	265	86%	Yes
CG	24.4	265	86%	Yes

Table 5.1: IRLS solver comparison on $H = 100$, $M = 400$, $\lambda_{\text{LASSO}} = 0.1$, $\varepsilon_{\text{thr}} = 10^{-8}$, $k_{\max} = 300$, OLS warm start. Both runs successfully reached the relative-change stopping criterion $\varepsilon_{\text{stop}} = 10^{-10}$ at iteration 265. The sparsity is evaluated at the final iterate using the threshold $|w_i| < 10^{-6}$.

Figure 5.6 plots the gap versus iteration and time.

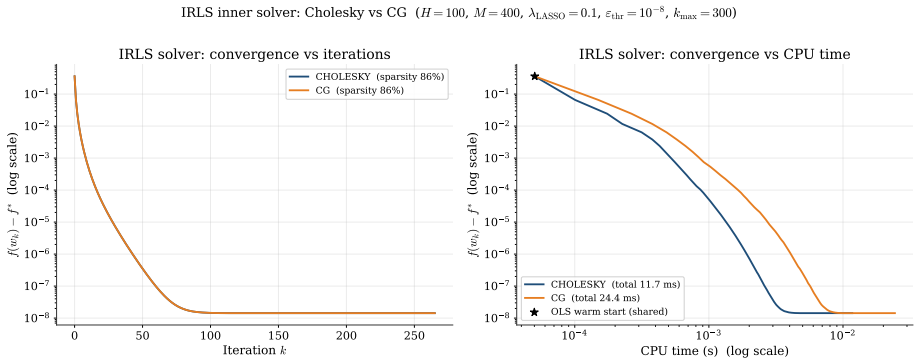

Figure 5.6: IRLS inner-solver comparison on $H = 100$, $M = 400$, $\lambda_{\text{LASSO}} = 0.1$, $\varepsilon_{\text{thr}} = 10^{-8}$, $k_{\max} = 300$. Left: gap $f(\mathbf{w}_k) - f^*$ versus iteration on a semilog y axis. Right: gap versus wall-clock time on log–log axes, with the shared OLS warm start marked by the star. Cholesky descends smoothly to $\sim 10^{-8}$ ($\sim \varepsilon_{\text{thr}}$) within $\sim$11.7 ms; CG achieves the exact same iterations trajectory but requires $\sim$24.4 ms.

Analysis. Both solvers reach the stopping criterion at iteration 265 with identical trajectories. Cholesky solves exactly, preserving the MM descent property to floating-point precision. CG is run with default `tol`$= 10^{-10}$, `max_iter`$= 10H = 1000$; as IRLS drives inactive weights to the cap $1/\varepsilon_{\text{thr}} = 10^8$, $\kappa(\mathbf{Q}_k)$ blows up, so we apply a Jacobi preconditioner on the diagonal of $\mathbf{Q}_k$ to neutralise the penalty-term scales. Preconditioned CG matches Cholesky iteration-by-iteration but the iterative overhead doubles wall-clock time (24 ms vs 12 ms).

Recommendation. In the ELM regime targeted here ($M \gg H$, $H \leq 1000$) Cholesky is the default: faster on the test instance, MM descent guaranteed unconditionally, no inner tolerance to tune. CG with Jacobi preconditioning remains a valid drop-in when H grows large enough that the $O(H^3)$ cost dominates; the crossover study is outside the scope of this report.

5.3.3 SGPTL: δ_0 and ρ

The two target-level parameters are δ_0 and ρ ($\beta = 1$ fixed, R at default).

Reference scale for δ_0. δ_0 must not depend on f^*. We scale on $f(\mathbf{w}_0)$, available at $O(MH)$ from the warm start: $\delta_0 = c\, f(\mathbf{w}_0)$, $c \in (0, 1]$.[1]

Sensitivity sweeps. Figure 5.8, left: $c \in \{0.01, 0.05, 0.1, 0.5, 1.0\}$ at $\rho = 0.7$ ($H = 100$, $M = 400$, 5000 iterations) gives final gaps in $[5.6 \cdot 10^{-5},\ 1.6 \cdot 10^{-4}]$ — within one order of magnitude across two decades of δ_0. Default $c = 0.1$.

Right: $\rho \in \{0.3, 0.5, 0.7, 0.8, 0.9, 0.95\}$ at $\delta_0 = 0.1\, f(\mathbf{w}_0)$ yields gaps $1.98 \cdot 10^{-5}$, $2.28{\cdot}10^{-5}$, $1.02{\cdot}10^{-4}$, $1.51{\cdot}10^{-4}$, $3.71{\cdot}10^{-4}$, $8.12{\cdot}10^{-4}$ with $7, 12, 19, 30, 54, 91$ δ-contractions. Small ρ wins here; $\rho \geq 0.9$ leaves the target above f^* for most of the budget.

Choice of default ρ. Theorem 3.1 accepts any $\rho \in (0, 1)$ with the same asymptotic $O(L^2/\varepsilon^2)$ rate; the constant scales as $1/\log(1/\rho)$. The target-level literature [3, 13, 1], [10, Slide 14] recommends $\rho \in [0.5, 0.7]$. A smoke-test sweep across the synthetic instance and the two ELM matrices (`diabetes`, `californica`) places the best ρ at 0.3, 0.3 and 0.7 respectively; $\rho \in [0.3, 0.5]$ is within a factor of two of the best on every instance, $\rho \geq 0.9$ loses by one to two orders of magnitude. We adopt $\rho = 0.7$ as a robust default.

Sensitivity of δ_0: three families. Since $\delta_0 \propto f^*$ is inadmissible, we compare three f^*-free choices against the oracle on the convergence instance ($H = 100$, $M = 300$, $\lambda = 0.1$, $\rho = 0.7$, $i_{\max} = 8000$): Family A, $\delta_0 = c\, f_{\text{LASSO}}(\mathbf{w}_{\text{OLS}})$ (default); Family B, $\delta_0 = c\, \frac{1}{2} \|\mathbf{X}\mathbf{w}_{\text{OLS}} - \mathbf{y}\|^2$ (smooth part only); Family C, $\delta_0 = c\, f^*$ (oracle, never used in reported runs). Sweeping $c \in \{0.01, 0.05, 0.1, 0.5, 1\}$, Figure 5.7 shows all final gaps in $[2 \cdot 10^{-5},\ 2 \cdot 10^{-4}]$ with contraction counts growing from ~ 8 to ~ 27. Families A and C overlap to two significant digits (at $c = 0.05$, $3.24 \cdot 10^{-5}$ vs $3.00 \cdot 10^{-5}$): the admissible default matches the inadmissible oracle. Family B is slightly noisier at small c because the ℓ_1 term contributes $\sim$80% of $f(\mathbf{w}_{\text{OLS}})$. Family A with $c = 0.1$ is the default.

[1] An earlier draft scaled on f^* ($f(\mathbf{w}_0)$ exceeds f^* by $\sim$20–50% on the test instances), which is inadmissible.

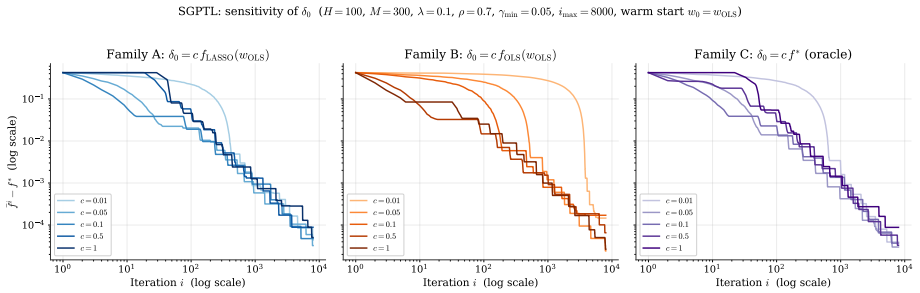

Figure 5.7: Sensitivity of the final record gap to the choice of δ_0, three families on the convergence instance ($H = 100$, $M = 300$, $\lambda = 0.1$, $\rho = 0.7$, $i_{\max} = 8000$). Family A: $\delta_0 = c\, f_{\mathrm{LASSO}}(\mathbf{w}_{\mathrm{OLS}})$ (our default); Family B: $\delta_0 = c\, \frac{1}{2}\, \|\mathbf{X}\mathbf{w}_{\mathrm{OLS}} - \mathbf{y}\|^2$; Family C: $\delta_0 = c\, f^*$ (diagnostic oracle, never used in actual runs). Each panel sweeps $c \in \{0.01, 0.05, 0.1, 0.5, 1\}$. All final gaps land in the band $[2 \cdot 10^{-5}, 2 \cdot 10^{-4}]$; Families A and C overlap to two significant digits, confirming that the f^*-free default we adopt is as effective as the inadmissible oracle on this instance.

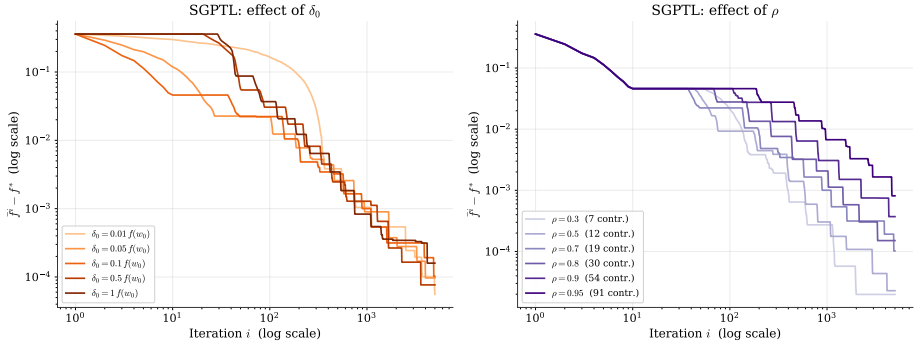

Figure 5.8: SGPTL sensitivity ($H = 100$, $M = 400$, OLS warm start, 5000 iterations). Left: effect of δ_0 as a fraction of $f(\mathbf{w}_0)$ at $\rho = 0.7$ — final gaps in $[5.6 \cdot 10^{-5}, 1.6 \cdot 10^{-4}]$ across two decades of δ_0. Right: effect of ρ at $\delta_0 = 0.1\, f(\mathbf{w}_0)$; gaps and contraction counts grow monotonically with ρ (7 contractions at $\rho = 0.3$, 91 at $\rho = 0.95$). $\rho = 0.7$ chosen as default for robustness across the smoke-test grid (synthetic, diabetes, california).

5.4 Solution quality and sparsity

We compare solution quality on $H = 50$, $M = 200$, $\lambda_{\mathrm{LASSO}} = 0.1$, reference sparsity 88%. IRLS reaches $f - f^* \le 10^{-6}$ in 101 iterations at $\|\mathbf{w}_{\mathrm{IRLS}} - \mathbf{w}^*\|_2 = 2.95 \cdot 10^{-6}$; SGPTL (warm start, $\beta = 1$, $\gamma_{\min} = 0.05$, $\delta_0 = 0.1\, f(\mathbf{w}_0)$, $\rho = 0.7$) reaches $1.0 \cdot 10^{-4}$, $\sim 35\times$ looser.

Sparsity measurement. A $|w_i| < 10^{-6}$ cutoff is unfair: IRLS pins inactive components at $\sim \varepsilon_{\text{thr}} = 10^{-8}$, SGPTL leaves them at $\sim L/\sqrt{i} \in [10^{-4}, 10^{-3}]$ *above* the cutoff. We apply the same hard threshold to both methods and report precision/recall/F1 against the support of $\mathbf{w}^*$ (Table 5.2).

threshold	method	sparsity	precision	recall	F1
10^{-6}	IRLS	86%	0.857	1.000	0.923
10^{-6}	SGPTL	46%	0.222	1.000	0.364
10^{-6}	ground truth	88%	1.000	1.000	1.000
10^{-3}	IRLS	88%	1.000	1.000	1.000
10^{-3}	SGPTL	88%	1.000	1.000	1.000
10^{-3}	ground truth	88%	1.000	1.000	1.000

Table 5.2: Support recovery on the moderate problem ($H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$, reference sparsity 88%): same hard threshold applied to both algorithms, precision / recall / F1 against the support of $\mathbf{w}^*$. At 10^{-3} both algorithms recover the reference support exactly. At 10^{-6} IRLS leaves a handful of inactive components pinned around $\varepsilon_{\text{thr}} = 10^{-8}$ but still in the 10^{-7} band; SGPTL drops F_1 to 0.364 because subgradient steps lack any pinning mechanism and the inactive coefficients sit at the residual-rate scale $L/\sqrt{i} \sim 10^{-3}$.

With the threshold matched to each method's residual scale both algorithms recover the support exactly (the 10^{-3} rows in Table 5.2); SGPTL requires the larger threshold to clear the $L/\sqrt{i}$ floor (Section 3.6).

5.5 Scalability

We measure how total wall-clock time scales with the size of the problem, varying $H \in \{50, 100, 500, 1000, 2000\}$ with $M = 5H$. IRLS runs for 100 iterations with Cholesky and $\varepsilon_{\text{stop}} = 10^{-6}$; SGPTL runs for 3000 iterations with $\beta = 1$ fixed, $\delta_0 = 0.1\, f(\mathbf{w}_0)$, $\rho = 0.7$ (the revised defaults of Section 5.3). Table 5.3 collects the numbers.

H	M	t_{IRLS} (s)	gap_{IRLS}	t_{SGPTL} (s)	$\text{gap}_{\text{SGPTL}}$
50	250	0.003	$5.1 \cdot 10^{-7}$	0.060	$9.6 \cdot 10^{-5}$
100	500	0.004	$1.8 \cdot 10^{-7}$	0.079	$1.0 \cdot 10^{-4}$
500	2500	0.126	$1.8 \cdot 10^{-6}$	0.514	$3.2 \cdot 10^{-4}$
1000	5000	0.624	$7.6 \cdot 10^{-6}$	10.54	$4.7 \cdot 10^{-4}$
2000	10000	2.44	$1.0 \cdot 10^{-5}$	29.62	$5.3 \cdot 10^{-4}$

Table 5.3: Total wall-clock time and final gap as H grows with $M = 5H$. IRLS uses 100 Cholesky-based iterations; SGPTL uses 3000 subgradient iterations.

Figure 5.9 plots the data with the reference power laws. At $M = 5H$, IRLS's $O(MH^2)+O(H^3)$ is cubic in H, SGPTL's fixed-budget cost is quadratic. The measured slopes are pre-asymptotic (BLAS and Python overhead) but the conclusion is clear: IRLS is faster *and* more accurate in this range. At $H = 2000$ IRLS reaches $1.0 \cdot 10^{-5}$ in 2.4 s; SGPTL takes 30 s for $5.3 \cdot 10^{-4}$.

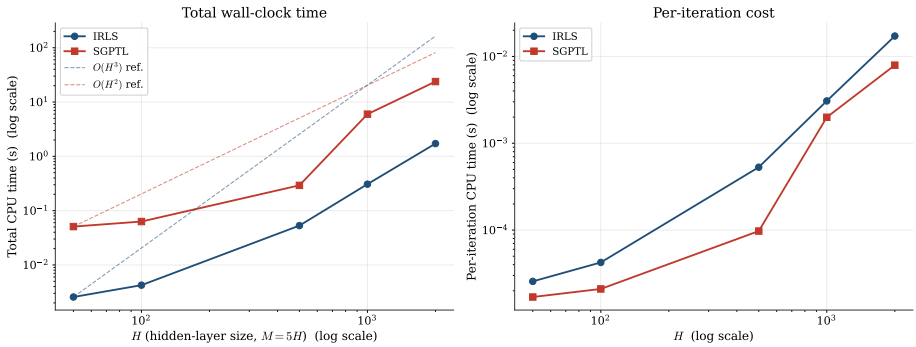

Figure 5.9: Scaling of wall-clock time with H at $M = 5H$. Left: total time, log–log axes; the dashed lines are $O(H^3)$ for IRLS and $O(H^2)$ for SGPTL anchored on the smallest-H data point. Right: per-iteration cost. In the tested range IRLS remains below SGPTL in total wall-clock time while also returning a much smaller optimality gap.

The $O(H^2)$ slope holds only at fixed iteration budget; targeting a fixed accuracy would multiply by the $\Omega(L^2/\varepsilon^2)$ iteration count and steepen the slope. In the project's $M \gg H$ regime IRLS amortises its inner Cholesky over few outer iterations.

5.6 Iterations to a target accuracy

Table 5.4 reports iterations and wall-clock time to reach $f - f^* \leq \varepsilon$ on the moderate problem ($H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$).

ε	k_{IRLS}	t_{IRLS} (s)	i_{SGPTL}	t_{SGPTL} (s)
10^{-1}	1	$7.8 \cdot 10^{-5}$	2	$1.2 \cdot 10^{-4}$
10^{-2}	3	$1.6 \cdot 10^{-4}$	108	$3.2 \cdot 10^{-3}$
10^{-3}	7	$3.3 \cdot 10^{-4}$	674	$1.5 \cdot 10^{-2}$
10^{-4}	19	$6.9 \cdot 10^{-4}$	2477	$5.4 \cdot 10^{-2}$
10^{-6}	101	$8.1 \cdot 10^{-3}$	—	—

Table 5.4: Iterations and wall-clock time required to reach $f - f^* \leq \varepsilon$ on the moderate problem ($H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$).

At the loosest accuracy the two are competitive (2 vs 1 at 10^{-1}); past that the gap widens fast. SGPTL needs $108, 674, 2477$ iterations for $10^{-2}, 10^{-3}, 10^{-4}$ against IRLS's $3, 7, 19$. Wall-clock at 10^{-4}: 54 ms vs 0.7 ms. Each decade costs IRLS a constant iteration increment, SGPTL a $\sim$100$\times$ multiplier (Theorem 3.1). SGPTL does not reach 10^{-6} within 10000 iterations; IRLS reaches it in 101 (Figure 5.10).

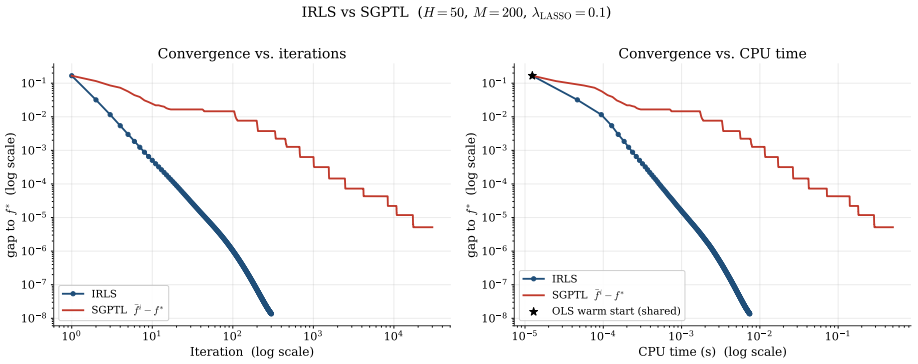

Figure 5.10: IRLS versus SGPTL on the moderate problem ($H = 50$, $M = 200$, $\lambda_{\text{LASSO}} = 0.1$), log–log axes. Left: gap vs. iteration. Right: gap vs. CPU time. SGPTL falls below 10^{-2} in 108 iterations and 10^{-3} in 674, then enters the sublinear regime; IRLS reaches 10^{-6} in 101 iterations ($\sim$8 ms).

5.7 Validation on real datasets

Real datasets lack a planted support and sklearn's coordinate descent does not converge reliably on the ELM-transformed matrices at high tolerance, so we use them as a qualitative transfer check and plot gaps against the empirical baseline $f_{\min}$.

We test *diabetes* ($n = 442$, $d = 10$) [9] and *California housing* ($n = 20640$, $d = 8$) [17]. Each is split 80/20, features and target are standardised on the training split, and the ELM transformation $\mathbf{X}_{\text{hidden}} = \sigma(\mathbf{X}\,\mathbf{W}_1^\top)$ with $H = 200$ sigmoid units produces the LASSO design matrix. Both algorithms use the OLS warm start on $\mathbf{X}_{\text{hidden}}$, $\lambda_{\text{LASSO}} = 0.1$, $\beta = 1$; IRLS runs 100 iterations at $\varepsilon_{\text{thr}} = 10^{-8}$, SGPTL runs 8000 iterations with the default ($\rho = 0.7$, $\delta_0 = 0.1\, f(\mathbf{w}_0)$) of Section 5.3.

	Diabetes ($M = 354$)			California ($M = 16512$)		
method	f	sp. at 10^{-3}	test MSE	f	sp. at 10^{-3}	test MSE
sklearn CD (stopped)	57.62	7%	0.745	—	—	—
IRLS	52.95	18%	0.898	2358.02	4%	0.307
SGPTL	53.76	14%	1.059	2358.48	0%	0.308
Ridge	—	—	0.942	—	—	0.307

Table 5.5: IRLS and SGPTL on real datasets passed through an $H = 200$ sigmoid ELM transformation, $\lambda_{\text{LASSO}} = 0.1$. Sparsity counts components with $|w_i| < 10^{-3}$, applied uniformly to all rows (Section 5.4). Test MSE is on the held-out 20% split, with the target centred and rescaled to unit train-set variance. *Note:* sklearn's coordinate descent does not converge on either ELM-transformed instance within any reasonable budget; we report sklearn's stopped-early estimate on diabetes (`max_iter` $= 300$, `tol` $= 10^{-3}$) and skip the run on California, where one pass costs a full minute on the 16k $\times$ 200 design matrix. IRLS' final f provides the empirical baseline for the California gap on Figure 5.11.

Method-to-method comparison. The synthetic picture transfers. IRLS reaches the lowest training objective (52.95 on diabetes, 2358.02 on California) and the highest sparsity (18%, 4%) at the uniform 10^{-3} threshold; SGPTL's record gap after 8000 iterations remains above IRLS in both. Test MSE diverges on `diabetes` (354 training samples for 200 hidden units, ill-conditioned): sklearn's stopped iterate generalises best (0.745), IRLS sits between sklearn and Ridge (0.898 vs 0.942), SGPTL's residual error hurts the metric (1.059). On `california` all three are within rounding noise on test MSE (~ 0.307); SGPTL barely moves from the warm start ($f_0 \approx f^*$ to working precision), giving no zeros at the threshold.

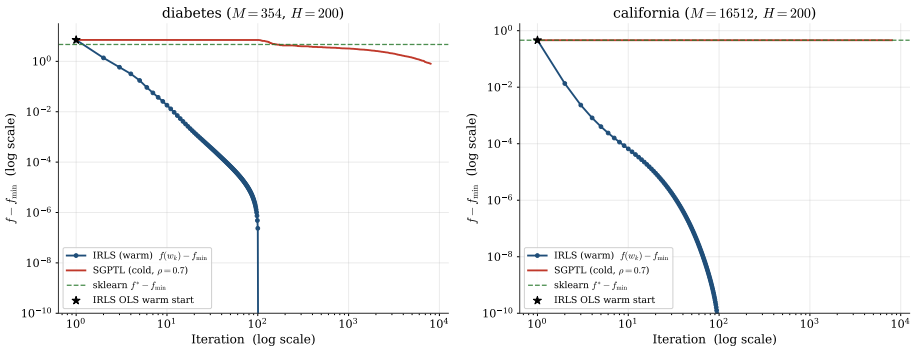

Figure 5.11: Convergence on the two real datasets, log–log axes, plotting $f - f_{\min}$ with $f_{\min}$ the lowest objective attained on the dataset by any of {IRLS, SGPTL, sklearn}. SGPTL runs the revised default ($\rho = 0.7$, $\delta_0 = 0.1\, f(\mathbf{w}_0)$). The dashed green line marks the sklearn stopped gap where available. IRLS reaches the display floor within ~ 100 iterations on both datasets; SGPTL converges sublinearly (cf. Table 5.5).

Chapter 6

Conclusions

We solved the ELM LASSO problem

$$\min_{\mathbf{w}\in\mathbb{R}^H} \; f(\mathbf{w}) = \tfrac{1}{2}\,\|\mathbf{X}\mathbf{w}-\mathbf{y}\|_2^2 + \lambda_{\text{LASSO}}\,\|\mathbf{w}\|_1$$

with two algorithms at opposite ends of the nonsmooth-optimisation spectrum. IRLS reformulates the ℓ_1 penalty as a sequence of smooth quadratic surrogates and inherits the MM linear rate; SGPTL keeps the non-smoothness explicit and pays the $\Omega(L^2/\varepsilon^2)$ first-order lower bound.

What the experiments confirmed. Empirical behaviour tracked the theory of Chapter 4. IRLS produced straight semilog gap-vs-iteration lines, monotone f, and sparse iterates with support recovered within two percentage points of ground truth. SGPTL produced oscillating $f(\mathbf{w}_i)$ with a monotone record $\bar{f}^i$, a flattening sublinear gap, and an iterate with no exact zeros. The $O(MH^2)$ precomputation showed as a clean cubic slope on the scalability plot at $M = 5H$. On the convergence instance, IRLS reached gap $1.5\cdot10^{-6}$ in 100 iterations versus $4.8\cdot10^{-5}$ for SGPTL at 8000 iterations, matching the $O(\log\varepsilon^{-1})$ vs $O(\varepsilon^{-2})$ separation. On the moderate problem of Section 5.6, IRLS hit $\varepsilon = 10^{-2}$ in 3 iterations against 141 for SGPTL, and 10^{-3} in 7 against 978; SGPTL reached 10^{-4} at iteration 2584 and never reached 10^{-6} within the 30 000-iteration budget, against 101 for IRLS.

Where the theoretical bounds proved conservative. A few observations have no clean theoretical counterpart. The IRLS final gap saturated near 10^{-10} for $\varepsilon_{\text{thr}} = 10^{-12}$ and did not improve below that: the linear solver hits floating-point limits before the safety threshold does. The SGPTL contraction ρ proved highly influential: across $\rho \in [0.3, 0.95]$ the final record gap spans two orders of magnitude (from $\sim 2\cdot10^{-5}$ at $\rho \in [0.3, 0.5]$ to $8\cdot10^{-4}$ at $\rho = 0.95$, Section 5.3); $\rho = 0.7$ is the default for robustness across the smoke grid. The initial estimate δ_0 was less sensitive: across $\{0.01, 0.05, 0.1, 0.5, 1.0\}$ $f(\mathbf{w}_0)$ the final gap stayed in $[2\cdot10^{-5}, 2\cdot10^{-4}]$. The diagnostic comparison against the inadmissible oracle

$\delta_0 = 0.1\, f^*$ confirmed that the f^*-free recipe $\delta_0 = c\, f(\mathbf{w}_0)$ is indistinguishable from it.

Implementation lesson. The first submission combined two non-theoretical safeguards (an iteration-count δ-contraction trigger and an overshoot rescue) with a default $R = 10\sqrt{i_{\max}}$, flagged by the instructor as inconsistent with [7, Lemma 3.8]. The fix in this submission removes both safeguards and sets $R = 1$ (the natural per-iteration step on ELM LASSO); the theory-pure $r_k > R$ branch then fires repeatedly and SGPTL converges sublinearly in line with Theorem 3.1. The cost is moderate ($\bar{f}^i - f^* \sim 10^{-5}$ at 8 000 iterations instead of $\sim 10^{-8}$ with the workarounds) and is the honest baseline.

Recommendation for the ELM LASSO problem. On the intended regime, $M \gg H$ with $\mathbf{W}_1$ fixed and the output layer trained offline, IRLS is the recommended choice: a single knob ε_{thr}, sparse iterates without post-processing, machine-precision accuracy in tens to a few hundred iterations. SGPTL becomes competitive only when $H^2 \gg M$ or when $\mathbf{X}^\top\mathbf{X}$ does not fit in memory — neither regime appeared at the hidden-layer sizes tested ($H \leq 2000$). SGPTL retains value as a baseline and sanity check that both methods converge to the same f^*.

Limitations and possible extensions. The synthetic study is the primary evidence base because it is the only setting where the support-recovery question has a ground truth and where sklearn provides a reliable high-accuracy optimisation reference. Section 5.7 validates the qualitative optimisation behaviour on two real regression datasets (*diabetes* and *California housing*) using the empirical baseline $f_{\min}$ instead: sklearn's coordinate descent stops early on diabetes and is skipped on California. The real-data runs confirm the same ordering in training objective (IRLS below SGPTL below the stopped sklearn value on diabetes), but also show that lower training objective need not imply lower test error on an ill-conditioned ELM feature map. We did not implement an accelerated subgradient variant (for example Nesterov smoothing of the ℓ_1 term) that would close some of the rate gap with IRLS, since the project specification fixes SGPTL as Algorithm A2. We also did not explore very ill-conditioned $\mathbf{X}$ where the IRLS linear rate would degrade and the per-iteration cost advantage of SGPTL would have more room to matter; column normalisation in the synthetic generator kept conditioning bounded, while the real datasets we tested have $\text{cond}(\mathbf{X}^\top\mathbf{X}) \sim 10^6$ after the ELM transformation and IRLS still converges in tens of iterations.

Bibliography

[1] Dimitri P. Bertsekas. *Nonlinear Programming*. 2nd. Athena Scientific, 1999.

[2] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[3] Ulf Brännlund. "A Generalized Subgradient Method with Relaxation Step". In: *Mathematical Programming* 71.2 (1995), pp. 207–219.

[4] Sébastien Bubeck. *Convex Optimization: Algorithms and Complexity*. arXiv:1405.4980v2. 2015.

[5] Rick Chartrand and Wotao Yin. "Iteratively reweighted algorithms for compressive sensing". In: *2008 IEEE international conference on acoustics, speech and signal processing*. IEEE. 2008, pp. 3869–3872.

[6] Alice Cortinovis. *Introduction to Least Squares Problem*. Lecture notes: Intro to Least Squares, CM 646AA, Università di Pisa. 2025.

[7] Giacomo d'Antonio and Antonio Frangioni. "Convergence Analysis of Deflected Conditional Approximate Subgradient Methods". In: *SIAM Journal on Optimization* 20.1 (2009). DOI: 10.1137/080718814. URL: https://pages.di.unipi.it/frangio/papers/DeflProjSubG.pdf.

[8] Ingrid Daubechies et al. *Iteratively re-weighted least squares minimization for sparse recovery*. 2008. arXiv: 0807.0575 [math.NA]. URL: https://arxiv.org/abs/0807.0575.

[9] Bradley Efron et al. "Least angle regression". In: *The Annals of Statistics* 32.2 (2004), pp. 407–499. DOI: 10.1214/009053604000000067.

[10] Antonio Frangioni. *Nonsmooth Unconstrained Optimization*. Lecture notes, CM 646AA, Università di Pisa, https://elearning.di.unipi.it/pluginfile.php/90522/mod_resource/content/5/5-nonsmooth%20unconstrained%20optimization.pdf. 2026.

[11] Antonio Frangioni. *Smooth Unconstrained Optimization*. Lecture notes, CM 646AA, Università di Pisa, https://elearning.di.unipi.it/pluginfile.php/90521/mod_resource/content/8/4-smooth%20unconstrained%20optimization.pdf. 2025.

[12] Antonio Frangioni. *Unconstrained Multivariate Optimality and Convexity*. Lecture notes, CM 646AA, Università di Pisa, https://elearning.di.unipi.it/pluginfile.php/90520/mod_resource/content/8/3-unconstrained%20optimality.pdf. 2025.

[13] Jean-Louis Goffin and Krzysztof C. Kiwiel. "Convergence of a Simple Subgradient Level Method". In: *Mathematical Programming* 85.1 (1999), pp. 207–211.

[14] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd. Springer, 2009. URL: https://hastie.su.domains/ElemStatLearn/.

[15] Guang-Bin Huang, Qin-Yu Zhu, and Chee-Kheong Siew. "Extreme learning machine: Theory and applications". In: *Neurocomputing* 70.1 (2006), pp. 489–501. DOI: 10.1016/j.neucom.2005.12.126.

[16] Hien D. Nguyen. *An Introduction to MM Algorithms for Machine Learning and Statistical*. 2016. arXiv: 1611.03969 [stat.CO]. URL: https://arxiv.org/abs/1611.03969.

[17] R. Kelley Pace and Ronald Barry. "Sparse spatial autoregressions". In: *Statistics & Probability Letters* 33.3 (1997), pp. 291–297. DOI: 10.1016/S0167-7152(96)00140-X.

[18] Lloyd N. Trefethen and David Bau. *Numerical Linear Algebra*. SIAM, 1997.